

JEEVAFLOW

Secure Healthcare Document Intelligence & Doctor-Ready Clinical Workflow

“From Scattered Patient Documents to Controlled, Source-Linked Clinical Review”

**WhatsApp is only the communication channel.
JeevaFlow is the controlled healthcare-data environment.**

PREPARED BY

Team Geek

CODEBASE

JeevaFlow API 2.0.0 (FastAPI) · React 19 / Vite
frontend

REPORT DATE

2 October 2026

STATUS

Hackathon MVP · development environment ·
synthetic data only

Contents

	<i>Executive Summary</i>	3
01	Executive Overview	3
02	Complete System Architecture	4
03	WhatsApp Security	6
04	Patient Authentication	7
05	Consent Management	8
06	Document Security	9
07	OCR and Information Extraction	10
08	Source Provenance	11
09	Doctor-Ready Form	12
10	Conflict Detection	14
11	Care Loops / Commitments	15
12	Doctor Authentication and MFA	16
13	Role-Based Access	17
14	Consent-Based Authorization	18
15	Evidence Security	19
16	Access Revocation	20
17	Audit Trail	21
18	Tamper Demonstration	22
19	Data Retention	22
20	Security Status	23
21	Frontend	24
22	Backend and Verified Technology Stack	25
23	API Architecture	26
24	Complete End-to-End Demo	28
25	Security & Test Validation	29
26	Security Threat Model	30
27	Security Design Principles	31
28	What JeevaFlow Does Not Do	31
29	Limitations / Development Status	32
30	Future Extensions	33
31	Final Security Summary	34
	<i>Appendix A. Audit Method and Evidence</i>	34

Executive Summary

JeevaFlow turns the medical documents that patients already send through messaging apps (prescriptions, lab reports, follow-up results) into a **consent-controlled, source-linked clinical view** that an authorized doctor can review in minutes.

Its core design rule is printed on every screen of the application:

WhatsApp is only the communication channel. JeevaFlow is the controlled healthcare-data environment.

What the audit verified:

AREA	VERIFIED STATE (2 OCT 2026)
Channel	Twilio-signed WhatsApp webhook; signature, replay, account, freshness and rate-limit checks; generic replies only
Patient trust	Transaction-bound 6-digit OTP (HMAC-stored, 5 min, single use, 5 attempts); explicit, scoped, time-limited consent
Documents	Validation → isolated worker (structural PDF scan, sanitise) → malware scan → AES-256-GCM vault → quarantine until OTP + consent
Clinical organisation	Deterministic, rule-based extraction; quote-or-reject provenance; timeline; conflict detection; care loops closed only by humans
Clinician access	Password (scrypt) + TOTP MFA; consent and scope re-checked on every request; single-use watermarked evidence views
Accountability	SHA-256 hash-chained, HMAC-signed, append-only audit trail with auditor verification and a tamper demonstration
Tests	156 automated backend tests passed and 8 frontend tests passed in the development environment; 13/13 live end-to-end checks passed

What JeevaFlow is not. It does not diagnose, prescribe, change medication or decide which of two conflicting values is correct. It organises documented information for authorized human review. It has **not** been certified against any regulation (HIPAA, DPDP or other), and several controls are explicitly demo-grade (Section 29).

1 Executive Overview

1.1 The problem

Patients carry their health history as loose PDFs, phone photos and forwarded messages. In practice this creates eight recurring problems:

PROBLEM	WHAT GOES WRONG
Fragmented information	Prescriptions, lab reports and follow-ups sit in different chats, phones and inboxes
Difficult clinical review	A doctor must open and read every file to reconstruct the history
Uncertain provenance	A typed summary cannot be traced back to the page it came from
Consent management	"I sent it on WhatsApp" is not a record of who may see what, for how long
Unauthorized access	Chat histories are forwarded, backed up and shared without control
Weak auditability	Nobody can say who viewed which document and when
Duplication and conflicts	The same test appears twice with different values; nobody notices
Long-term retention risk	Medical files stay in chat backups indefinitely

Instructions such as *"Repeat HbA1c after 3 months"* are written once and then forgotten, because nothing tracks whether they happened.

1.2 The JeevaFlow approach

JeevaFlow keeps the convenience of messaging for the **interaction**, but moves the **data** into a controlled environment:

1. **Receives** documents through a communication channel (WhatsApp via Twilio) or the patient portal.
2. **Validates and sanitises** them in an isolated worker process, then scans them.
3. **Authenticates** the patient with a one-time code bound to a transaction.
4. **Obtains explicit consent** naming the doctor, the data scopes and the duration.
5. **Stores** each document encrypted with its own key, quarantined until verified.
6. **Extracts** lab values, instructions, medications and allergies with deterministic rules.
7. **Preserves source evidence**: every item keeps its exact quote, page and position.
8. **Builds a patient timeline** ordered by the report date found in each document.
9. **Identifies conflicts and care loops**: disagreeing values and unfinished instructions.
10. **Presents a doctor-ready form** to the consented doctor only, after password + TOTP sign-in.
11. **Provides auditable evidence access** through single-use, watermarked page views.
12. **Supports revocation and lifecycle controls**: consent revocation, document deletion by key destruction and automatic retention expiry.

The guiding phrase in the README is “*AI reads. Rules validate. Humans decide.*” In the current codebase the reading is done by **text-layer extraction and local OCR**, not by a generative-AI model (Section 7).

2 Complete System Architecture

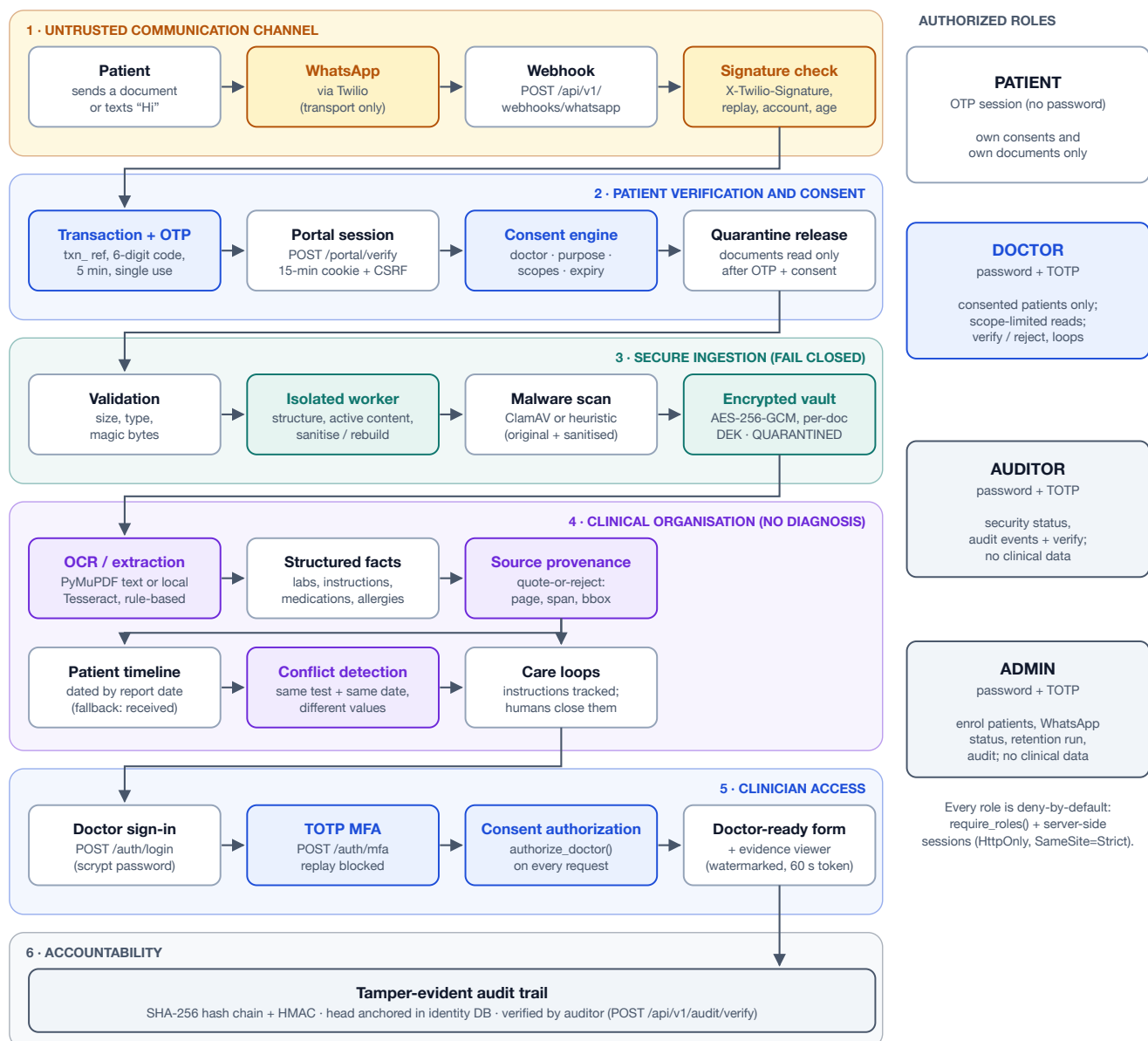
The full architecture is shown in Figure 2.1 after the component table.

2.1 Components

#	COMPONENT	WHAT IT DOES	CODE
1	Patient	Sends a document or texts “Hi” from an enrolled WhatsApp number	—
2	WhatsApp / Twilio	Transport only; Twilio signs every webhook call	external
3	Webhook	<code>POST /api/v1/webhooks/whatsapp</code> ; body cap, per-IP rate limit, HTTPS required in production	<code>routes/webhook.py</code>
4	Signature verification	<code>X-Twilio-Signature</code> HMAC-SHA1 check, constant-time, fails closed (503 if not configured, 403 if invalid)	<code>whatsapp.py</code>
5	Transaction + OTP	<code>txn_<random></code> binds patient, OTP, consent and documents; expires after 30 min	<code>security/otp.py</code>
6	Portal session	<code>POST /api/v1/portal/verify</code> issues a 15-minute HttpOnly session and a CSRF token	<code>routes/portal.py</code> , <code>security/auth.py</code>
7	Consent engine	Doctor, purpose, scopes, 1/7/30-day expiry; revoke or end now	<code>security/consent.py</code>
8	Secure ingestion	Size/type/magic-byte validation, isolated inspection and sanitisation, malware scan, SHA-256 dedupe	<code>pipeline.py</code> , <code>storage.py</code>
9	Validation / quarantine	New documents are stored as <code>QUARANTINED</code> ; nothing is read until OTP and consent	<code>pipeline.receive_document()</code>
10	Encrypted vault	AES-256-GCM per document with a wrapped data key; crypto-shredding on delete	<code>vault.py</code> , <code>security/crypto.py</code>
11	OCR / extraction	PDF text layer or local Tesseract, in a fresh isolated process	<code>worker.py</code> , <code>extraction.py</code> , <code>facts.py</code>
12	Structured facts	Observations, instructions (commitments), medications, allergies, prescriber	<code>models.py</code>
13	Source provenance	Quote-or-reject anchoring with page, character span, bounding box, confidence	<code>provenance.py</code>
14	Patient timeline	Documents, observations and instructions ordered by report date	<code>timeline.py</code>
15	Conflict detection	Same test, same date, different values → human review	<code>conflicts.py</code>
16	Care-loop tracking	Instructions tracked as Open Loops; system suggests, humans close	<code>loops.py</code> , <code>matching.py</code>
17	Doctor authentication	Username + script password, then TOTP	<code>routes/auth.py</code> , <code>security/auth.py</code>

#	COMPONENT	WHAT IT DOES	CODE
18	Consent-based authorization	<code>authorize_doctor()</code> on every patient-data request	<code>security/consent.py</code>
19	Doctor-ready form	Consent-scoped clinical summary with evidence links	<code>doctor_brief.py</code>
20	Evidence access	Single-use 60-second token → watermarked page render	<code>routes/doctor.py</code> , <code>worker.py</code>
21	Audit trail	Hash-chained, HMAC-signed, append-only events; auditor verification	<code>security/audit.py</code>

Two supporting stores keep identities apart from medical data: `identity.db` holds names, encrypted phone numbers (with an HMAC blind index), staff accounts, sessions and the audit anchor. `jeevaflow.db` knows patients only by a random `pt_...` reference.



WhatsApp transports the interaction; JeevaFlow controls the healthcare data. Nothing clinical is ever sent back over WhatsApp.

Modules: `routes/webhook.py` · `whatsapp.py` · `security/otp.py` · `security/consent.py` · `pipeline.py` · `worker.py` · `vault.py` · `extraction.py` · `facts.py` · `provenance.py` · `timeline.py` · `conflicts.py` · `loops.py` · `matching.py` · `security/auth.py` · `routes/doctor.py` · `doctor_brief.py` · `security/audit.py` · `retention.py`

Figure 2.1 — End-to-end architecture. Boxes name the real endpoints and functions. Roles on the right are the four authenticated principal types found in `identity.py` (Role).

3 WhatsApp Security

WhatsApp transports the interaction; JeevaFlow controls the healthcare data.

WhatsApp (through Twilio) is treated as an **untrusted transport**. It authenticates nothing on JeevaFlow's behalf and grants no access: being able to send a WhatsApp message only ever leads to a *verification request*, never to data.

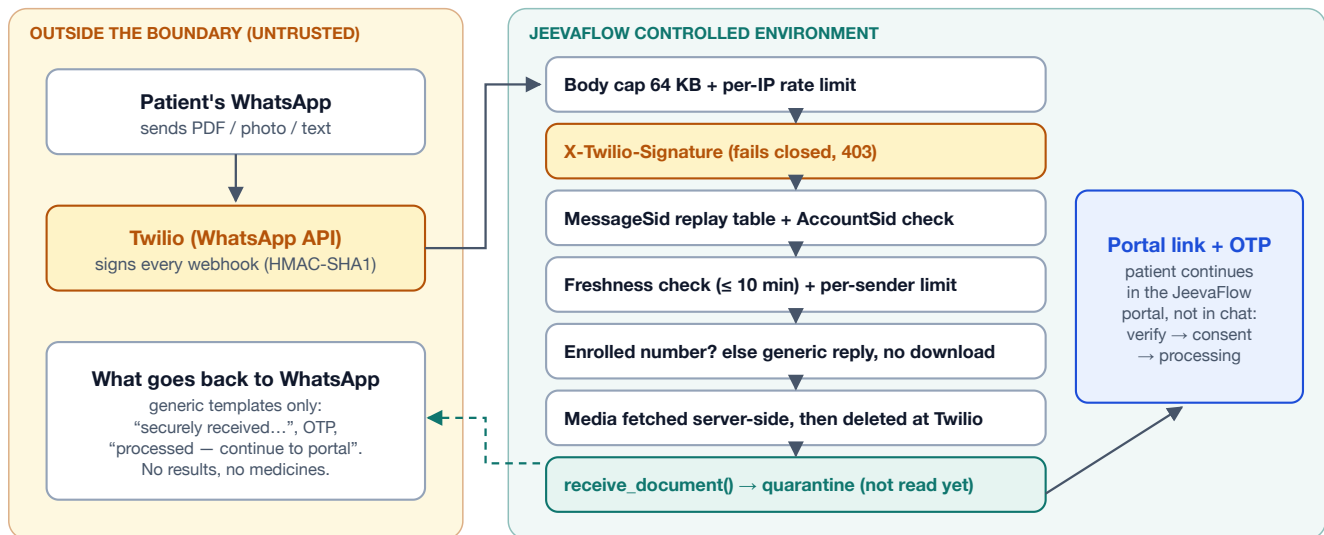


Figure 3.1 — Checks applied, in order, inside `process_webhook()` and `handle_inbound_message()`.

3.1 What is implemented

CONTROL	IMPLEMENTATION	EVIDENCE
Signature	<code>X-Twilio-Signature</code> = HMAC-SHA1 over URL + sorted parameters, compared in constant time; the public URL is configurable so signatures verify behind a tunnel	<code>test_unsigned_and_wrongly_signed_requests_are_rejected</code>
Fail closed	No signing token configured → 503; invalid signature → 403 and <code>WEBHOOK_SIGNATURE_REJECTED</code> audit	<code>test_webhook_fails_closed_without_signing_token</code>
Replay	<code>MessageSid</code> stored in a processed-message table; a repeat is ignored; malformed SIDs rejected	<code>test_replayed_message_sid_is_ignored</code>
Account / freshness	<code>AccountSid</code> must match; message age checked against Twilio's record (10-minute window)	<code>test_account_mismatch_rejected</code> , <code>test_stale_message_is_dropped</code>
Rate limits	120 requests/min per IP; 20 messages/10 min per sender; 64 KB body cap	<code>test_sender_rate_limit</code>
Unknown senders	Generic reply; media is never downloaded ; no patient is created	<code>test_unenrolled_number_gets_generic_reply_and_no_download</code>
Media handling	Fetched server-side; credentials sent only to <code>https://*.twilio.com</code> ; size and status enforced; deleted from Twilio afterwards	<code>test_download_sends_credentials_only_to_twilio</code> , <code>test_media_deletion_targets_only_twilio_media</code>
Ingestion	Media goes through <code>receive_document()</code> and stays quarantined until OTP + consent	<code>test_media_is_quarantined_until_opt_and_consent</code>

3.2 Generic replies

All replies come from a fixed template set (`REPLIES` in `whatsapp.py`). Examples verified live:

- “Your document has been securely received. Verification required: open {link} and enter the code from the next message.”
- “Your JeevaFlow verification code is •••••. It expires in 5 minutes. Never share this code with anyone.”

- “Your document has been processed. Continue to the verified JeevaFlow portal.”

No reply ever contains a diagnosis, result, medicine, prescription text or preview

(`test_replies_contain_no_medical_information`). TwiML output is XML-escaped (`test_twiml_is_escaped`).

3.3 Demo simulator

`POST /api/v1/demo/whatsapp` builds a Twilio-style form, signs it with the configured signing token and sends it through the **real** webhook code path. Only the Twilio media download is replaced by the local synthetic file; media deletion, freshness and outbound notifications are reported as `SIMULATED` when Twilio credentials are absent, which was the case for this audit. A **Forged signature** button flips one character of the signature to show the 403 (`test_tampered_demo_signature_is_rejected`).

4 Patient Authentication

Patients never create a password. Every portal session starts from a **transaction** created by the webhook for an enrolled number, and is unlocked by a **one-time passcode** delivered to that number.

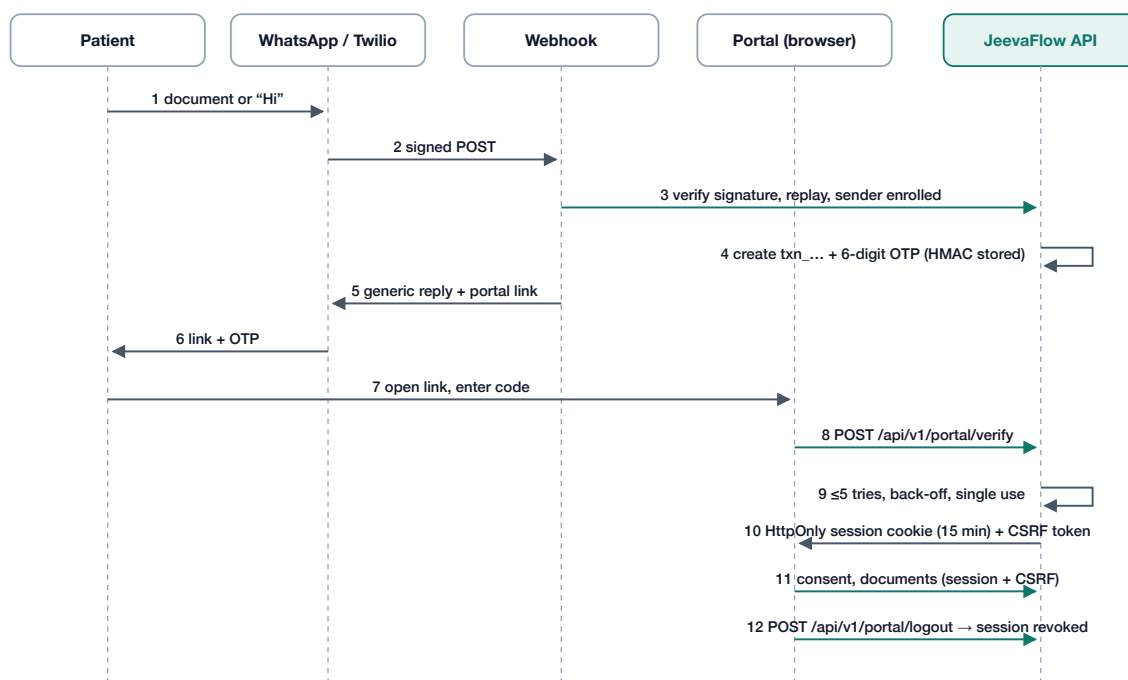


Figure 4.1 — Patient → WhatsApp → secure link → JeevaFlow portal → OTP → verified session.

4.1 OTP rules (security/otp.py)

RULE	VALUE
Code	6 digits from a CSPRNG (<code>secrets.randbelow</code>)
Validity	5 minutes, single use
Storage	Only <code>HMAC-SHA256(server secret, otp + transaction ref)</code> ; plaintext never stored or logged
Attempts	At most 5, with exponential back-off between attempts (≤ 30 s); the transaction locks after the 5th failure
Comparison	Constant-time; an HMAC is computed even for unknown transactions so timing does not reveal valid references
Errors	One generic message: “Verification failed or expired.”
Transaction	<code>txn_<random></code> (≈128-bit), expires after 30 minutes; expired/locked transactions trigger shredding of their unverified documents

4.2 Session lifecycle

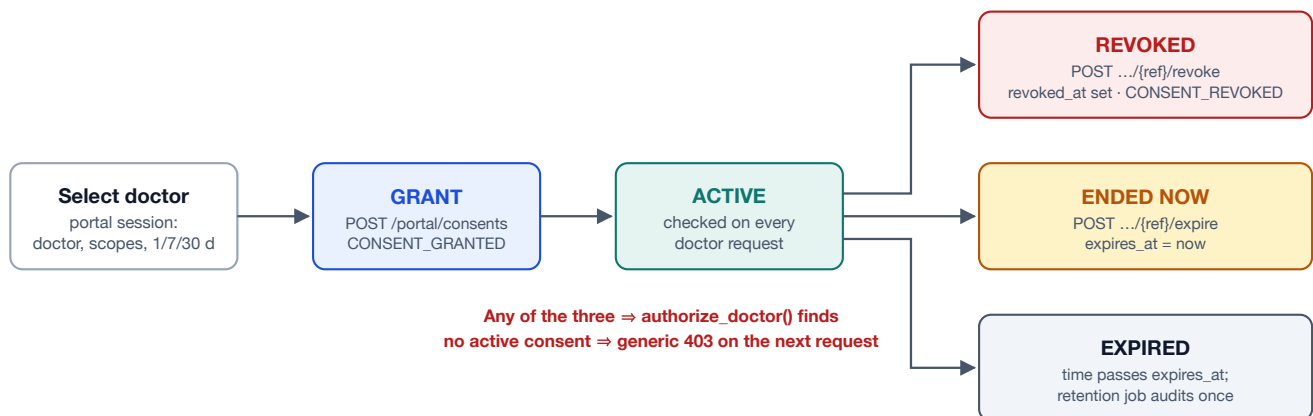
- **Verify:** `POST /api/v1/portal/verify` revokes any earlier portal session for the patient (one live session per patient), then creates a server-side session.

- **Session:** random 32-byte token in an `HttpOnly`, `SameSite=Strict` cookie scoped to `/api`; the database stores only an HMAC of the token; lifetime **15 minutes**.
- **CSRF:** every state-changing request must send `X-CSRF-Token`, derived from the session; the frontend keeps it in memory only.
- **Logout:** `POST /api/v1/portal/logout` revokes the session server-side and clears the cookie.

Why OTP. The OTP proves possession of the registered phone *at the time of the request* without asking patients to manage credentials, and binds that proof to one transaction. A forwarded portal link alone is useless without the code; a leaked code alone is useless without the transaction reference; and both expire quickly. Tests: `test_incorrect_otp_and_generic_error`, `test_otp_brute_force_locks_transaction`, `test_otp_backoff_blocks_rapid_retries`, `test_otp_is_single_use`, `test_expired_otp`, `test_otp_never_stored_in_plaintext`, `test_transaction_refs_are_random_and_not_sequential`.

5 Consent Management

Consent in JeevaFlow is **explicit, scoped, time-limited, revocable and enforced on the server**. It is created by the verified patient in the portal, never by a doctor and never implied by sending a message.



There is no doctor-initiated “request” state in the code: consent is patient-initiated from the verified portal. Purpose is fixed to `CLINICAL_REVIEW`.

Figure 5.1 — Consent states as implemented (`consent_status()` : `ACTIVE`, `REVOKED`, `EXPIRED`).

5.1 What a consent contains

FIELD	VALUES IN CODE
Doctor	One active staff user with role <code>DOCTOR</code> , chosen from a list
Purpose	<code>CLINICAL_REVIEW</code> (the only purpose currently defined)
Scopes	Any of <code>DEMOGRAPHICS</code> , <code>MEDICATIONS</code> , <code>LABS</code> , <code>ALLERGIES</code> , <code>INSTRUCTIONS</code> , <code>SOURCE_DOCUMENTS</code> (at least one)
Duration	1, 7 or 30 days
Lifecycle	<code>granted_at</code> , <code>expires_at</code> , <code>revoked_at</code> ; audited as <code>CONSENT_GRANTED</code> , <code>CONSENT_REVOKED</code> , <code>CONSENT_EXPIRED</code>

About “request”: the code has no doctor-initiated consent *request* state. The lifecycle is *select doctor* → *grant* → *active* → *revoked* / *ended now* / *expired*.

5.2 Enforcement properties

- **Tied to access:** documents in a transaction are only decrypted and read *after* consent is granted (or an existing active consent is applied with `POST .../consents/{ref}/apply`).
- **Checked per request:** every doctor route calls `authorize_doctor()`, which queries for a non-revoked, non-expired consent from *this* patient to *this* doctor for *this* purpose, covering the scope being read.
- **Immediate revocation:** `POST .../revoke` sets `revoked_at`; `POST .../expire` (“End now”) sets `expires_at` to now. Either way the very next doctor request fails.
- **Expiry:** once `expires_at` passes, access stops; the retention job records `CONSENT_EXPIRED` once.

5.3 Demonstrated behaviour

A doctor only sees patients who have granted that doctor an active consent. Live run: the synthetic patient granted consent to *Dr. Example*. *Dr. Example*’s patient list contained exactly that patient; *Dr. Other*, with no consent, saw an empty list and received `403` when requesting the same patient’s form directly. The rule is symmetric: had the patient consented to *Dr. Other* instead, the outcome would be reversed.

CONSENT STATE	DOCTOR	RESULT
Active consent to Dr. Example	Dr. Example	Patient listed; form, journey, loops, conflicts available within scopes
No consent	Dr. Other	Empty list; direct request → generic 403

This matters because the alternative (any authenticated doctor can open any patient) turns a single stolen staff login into a breach of every record. With consent-bound access, the blast radius is limited to the patients who chose that doctor, for the scopes and duration they chose. Tests: `test_doctor_without_consent_is_denied`, `test_wrong_doctor_is_denied`, `test_expired_consent_denied`, `test_patient_can_expire_consent_now`, `test_doctor_form_respects_scope`.

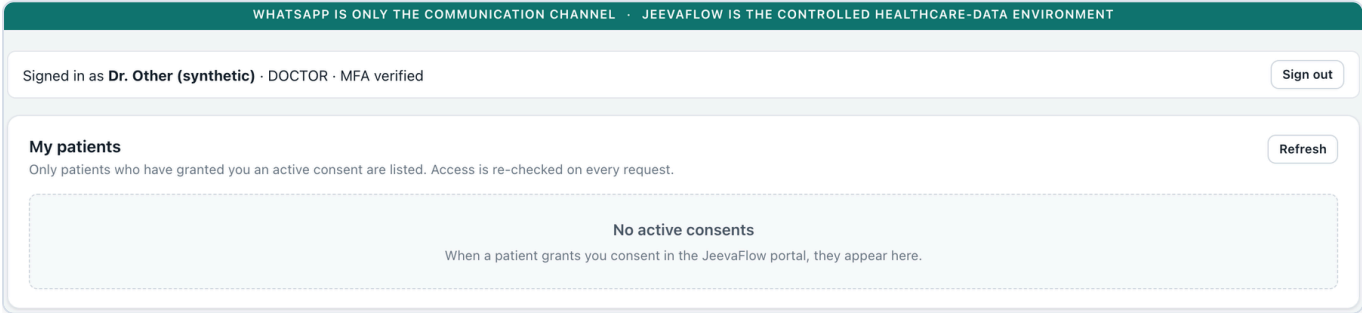


Figure 5.2 — Dr. Other (MFA-verified) has no active consents and therefore sees no patients.

6 Document Security

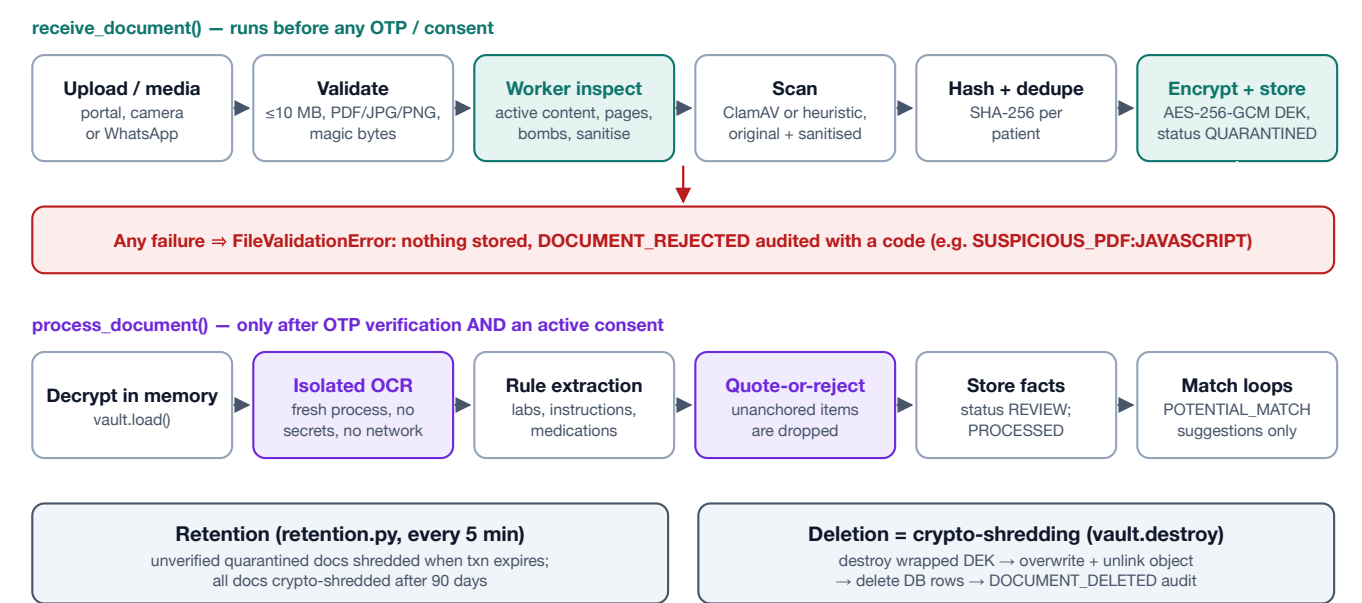


Figure 6.1 — `receive_document()` secures a file before anyone is verified; `process_document()` reads it only after OTP and consent.

6.1 Intake and validation

- **Formats:** PDF, JPEG, PNG only; up to **10 MB** (configurable) and **30 PDF pages**.
- **First line (`storage.py`, no parsing):** empty file, size, declared type and **magic bytes** must agree (`CONTENT_MISMATCH` otherwise).

- **Isolated worker (`worker.py`)**: all parsing of untrusted bytes happens in a separate process (Section 7.1). For PDFs it rejects encrypted files, bad page geometry and too many pages, and runs a **structural active-content scan** (`security/pdf_active_content.py`). The scan walks the parsed object tree and rejects JavaScript, non-navigation `OpenAction` , `/AA` additional actions, Launch, SubmitForm/ImportData/GoToR/GoToE, rich media and XFA, and embedded files. Unparseable structures fail closed. An inert C2PA provenance manifest is the only attachment accepted, and sanitisation strips it.
- **Sanitisation**: PDFs are scrubbed (metadata, links, JavaScript, attachments, form data, thumbnails) and rebuilt; the rebuilt copy is re-scanned and must contain no attachments. Images are re-encoded, which drops EXIF/GPS metadata and any appended payload; decompression bombs and animated images are rejected.
- **Malware scan (`security/scanner.py`)**: ClamAV (`clamscan` / `clamscan`) when installed, otherwise a heuristic scanner (EICAR, embedded PE/ELF/Mach-O/ZIP signatures, script in images, PDF action keywords). Both the original and the sanitised bytes are scanned; scanner errors fail closed. **In this environment the heuristic engine was active** (ClamAV not installed). This is reported as *DEMO IMPLEMENTATION* by the security dashboard.
- **Integrity**: SHA-256 of the original is recorded and used for per-patient deduplication; the sanitised copy's hash is stored separately.

Rejected files are **never stored**: the upload fails with a user-safe message and a `DOCUMENT_REJECTED` audit event carrying only a code (for example `SUSPICIOUS_PDF:JAVASCRIPT`).

6.2 Quarantine

Accepted documents are encrypted and stored with status `QUARANTINED` . Nothing is extracted from them until the patient has verified the OTP **and** an active consent exists (`test_media_is_quarantined_until_opt_and_consent` , `test_portal_upload_requires_active_consent`). If the transaction expires or locks first, the retention job crypto-shreds them (`test_expired_unverified_transaction_documents_are_shredded`).

6.3 Encrypted storage

LAYER	IMPLEMENTATION (<code>SECURITY/CRYPTO.PY</code> , <code>VAULT.PY</code>)
Documents	Envelope encryption: random 256-bit data key (DEK) per document , AES-256-GCM, unique nonce, document reference as associated data
Key wrapping	DEK wrapped by a key-encryption key (KEK) through a <code>KeyProvider</code> interface; the current provider reads the KEK from the environment
Database fields	Medical text columns encrypted with AES-256-GCM using an HKDF-derived key; identity columns use a separate identity key in a separate database
Phone lookup	HMAC-SHA256 blind index; no plaintext phone number stored
Binding	A ciphertext cannot be swapped between documents (<code>test_ciphertext_cannot_be_swapped_between_documents</code>)

6.4 Deletion and identifiers

Deletion is crypto-shredding (`vault.destroy()`): the wrapped DEK is destroyed first, then the ciphertext is overwritten and unlinked, then database rows are removed and `DOCUMENT_DELETED` is audited (`test_delete_document_destroys_key_object_and_rows`). Patients can delete their own documents from the portal.

Identifiers: patients, documents, consents, transactions, facts and staff use opaque random references (`pt_...` , `doc_...` , `cns_...` , `txn_...` , `fct_...` , `usr_...` , ~128 bits). Observation, commitment and evidence records in the doctor API use **integer IDs**. These are protected by the same consent check and an identical 403 for unknown and unauthorized objects (`test_unknown_and_foreign_objects_look_the_same`), but they are not opaque (see Section 29).

7 OCR and Information Extraction

Honest classification. Extraction in JeevaFlow is **deterministic and rule-based** (regular expressions, a small formulary, plausibility ranges). Text comes from the PDF text layer (PyMuPDF) or from **Tesseract OCR 5.5.3** running locally. Tesseract's recogniser is a trained LSTM model: that is the only machine-learned component, and it is ordinary OCR, not generative AI. **No LLM is used, and no document or extracted value is sent to any external AI service** (the security dashboard reports "External AI: DISABLED").

7.1 Isolated processing worker

Every inspect, extract and render job runs in a **fresh Python process** (`worker_client.py` → `worker.py`), one job at a time. The worker:

- gets a minimal environment with no secrets;
- has no database access and no keys;
- has network egress blocked in-process;
- runs with core dumps disabled and a 120-second CPU limit;
- refuses to run as root;
- writes only to a private temporary directory that is deleted after the job.

Worker error text is never echoed, because it could contain document content. Tests:

`test_worker_environment_has_no_secrets`, `test_worker_blocks_network`.

7.2 Pipeline

STEP	WHAT HAPPENS	CODE
Text	PDF pages with a text layer use PyMuPDF (<code>get_text()</code> (TEXT))	<code>worker.op_extract</code>
OCR	Pages without text are rendered at 200 DPI and passed to Tesseract (OCR); word boxes and confidences are kept	<code>worker.ocr_words</code>
Image quality gate	Photos are checked for size, sharpness (variance of the Laplacian, OpenCV) and brightness → <code>GOOD</code> , <code>WARN</code> or <code>RETAKE</code> ; <code>RETAKE</code> images are not interpreted	<code>quality.py</code>
Document date	“Report Date” / “Date” labels, day-first (DD/MM/YYYY); falls back to date received	<code>extraction.extract_document_date</code>
Lab observations	HbA1c, Glucose, Creatinine, LDL Cholesterol, Blood Pressure, with plausibility ranges that only reject obvious misreads	<code>extraction.OBSERVATION_PATTERNS</code>
Instructions	Lines starting with <i>repeat</i> , <i>follow-up with/in/after</i> , <i>review</i> , <i>refer to</i> , <i>recheck</i> ; due date derived from a stated interval (“after 3 months”)	<code>extraction.COMMITMENT_PATTERNS</code>
Medications	Tab/Cap/Syp/Inj lines → name, dose, unit, frequency, duration; 20-entry reference formulary with dose ranges	<code>facts.extract_medications</code>
Allergies, prescriber	Allergy lines (incl. “NKDA”); prescriber name and registration number	<code>facts.py</code>

7.3 Normalisation and fact states

Each medication field carries one of four states:

STATE	MEANING	EXAMPLE (SYNTHETIC PRESCRIPTION)
<code>SOURCE_FACT</code>	Read literally from the text	Atorvastatin 10 mg, “once daily at night”, 30 days
<code>AI_INFERRED</code>	A rule-based normalisation of source text; the original text is kept alongside	“1-0-1” → twice daily; “BD” → twice daily
<code>UNCERTAIN</code>	Unknown medicine, implausible dose/unit/duration, conflicting statements, or OCR confidence < 0.80	“Zyxorin 50 mg” (not in the formulary)
<code>MISSING</code>	Not present in the source; value left empty, never invented	Glimepiride 1 mg: frequency and duration

Despite its name, `AI_INFERRED` is produced by deterministic lookup rules, not by a model. Validation can only **lower** confidence; it never adds data (`test_never_invents_frequency_or_duration`, `test_unknown_medicine_is_uncertain`, `test_low_ocr_confidence_downgrades_to_uncertain`).

8 Source Provenance

Provenance is JeevaFlow’s core safety property: **an extracted item exists only if the exact text it came from can be found in the document.**

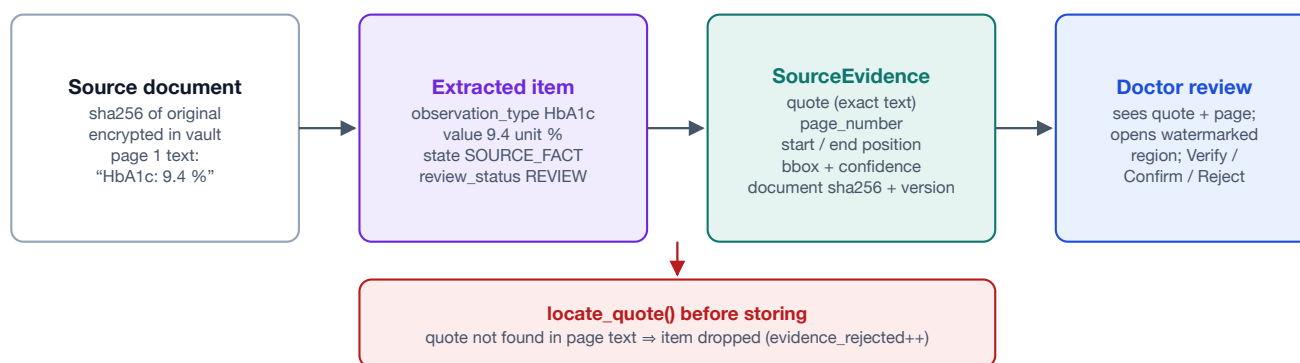


Figure 8.1 — SOURCE DOCUMENT → EXTRACTED FACT → SOURCE REFERENCE → DOCTOR REVIEW.

For every observation, instruction and clinical fact, a `SourceEvidence` row stores:

- the **exact quote**, page number and start/end character positions;
- the **bounding box** on the page (when locatable) and a confidence;
- the **document SHA-256** and the **pipeline version**.

Before anything is stored, `locate_quote()` re-checks the quote against the page text. **Items that cannot be anchored are dropped**, not stored (`evidence_rejected` in the response; `test_unanchored_extraction_is_rejected`, `test_every_fact_has_an_exact_quote`, `test_provenance_quote_or_reject`). The pipeline also re-checks quotes returned by the worker, which protects against a misbehaving worker.

All extracted observations start with review status **REVIEW**; only a doctor can mark them **VERIFIED** or **REJECTED**, and verification is refused for an item without evidence.

Medicine	Dosage	Frequency	Duration	Status	Evidence
Metformin	500 mg	twice daily AI-inferred Normalised from "1-0-1"	30 days	AI-inferred Confirm Reject	Source Document received via WhatsApp, p.1
“ 1. Tab Metformin 500 mg 1-0-1 x 30 days Document received via WhatsApp · report 20 Sep 2026 · page 1 · confidence 100% View source region					

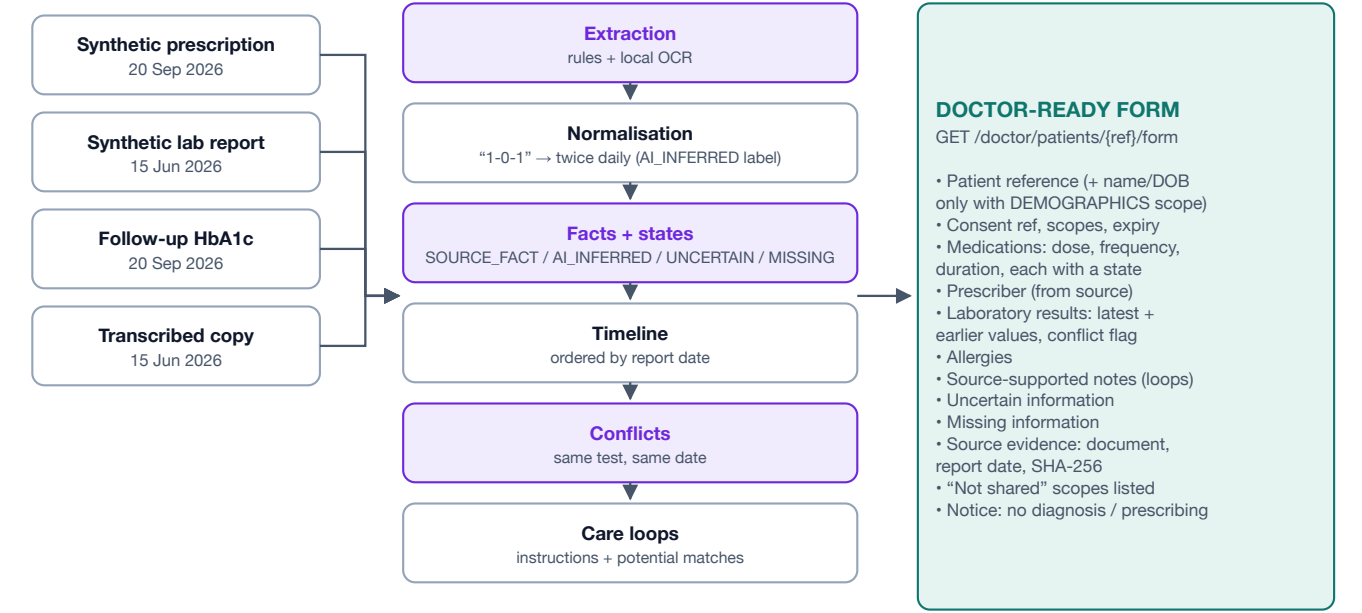
Figure 8.2 — In the doctor form, “Source” expands the exact quote, document, report date, page and confidence for each item.

Why it matters:

- **Traceability:** every value points to a page.
- **Clinical review:** the doctor checks the source, not a paraphrase.
- **Correction:** wrong items are rejected, and dependent suggestions are withdrawn.
- **Accountability:** every decision is attributed and audited.
- **Reduced ambiguity:** “where did this number come from?” always has an answer.

9 Doctor-Ready Form

The doctor-ready form (`GET /api/v1/doctor/patients/{patient_ref}/form`, built by `doctor_brief.py`) is the point of the whole pipeline. It turns four scattered synthetic documents into **one consent-scoped, source-linked page**.



Scopes not covered by the active consent are omitted server-side (doctor_brief.py), not hidden in the UI.

Figure 9.1 — From scattered documents to the doctor-ready form.

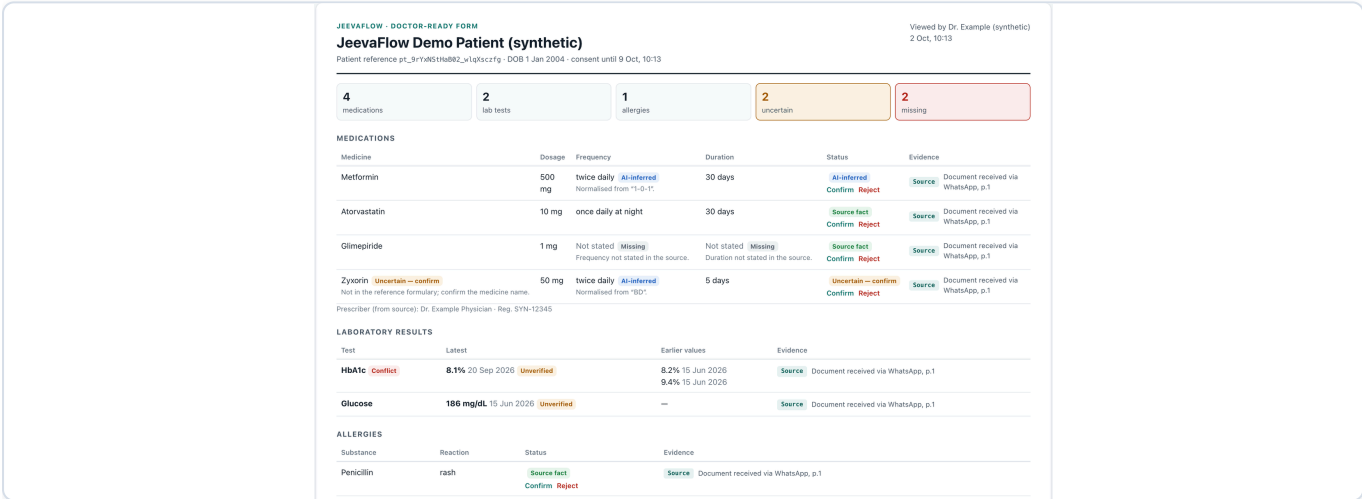


Figure 9.2 — Live doctor-ready form (synthetic patient): summary tiles, medications with per-field states and Confirm/Reject, laboratory results with conflict flag, allergies, each with a Source link.

9.1 Contents (as implemented)

SECTION	CONTENT	SCOPE REQUIRED
Patient	Reference; name and date of birth only if shared	DEMOGRAPHICS
Consent	Reference, purpose, scopes, expiry, whether source viewing is allowed; scopes not shared are listed	—
Medications	Name, dose, frequency, duration, each field with its state; prescriber from source	MEDICATIONS
Laboratory results	Per test: latest value, earlier values, dates, review status, conflict flag	LABS
Allergies	Substance, reaction, state	ALLERGIES
Source-supported notes	Instructions / Open Loops with state, due date and documented date	INSTRUCTIONS
Uncertain information	Every UNCERTAIN field and every conflicted test	per section
Missing information	Every MISSING field (e.g. no frequency stated)	per section
Source evidence	Each processed document: label, report date, received time, pages, SHA-256, scan engine	—

SECTION	CONTENT	SCOPE REQUIRED
Controls	Confirm / Reject facts; Verify / Reject observations; view source region	—

Scopes outside the active consent are **omitted by the server**, not hidden by the UI (`test_doctor_form_respects_scope` , `test_instruction_scope_does_not_reveal_lab_values`). Rejected items are left out. Values are shown as written; nothing is labelled better or worse.

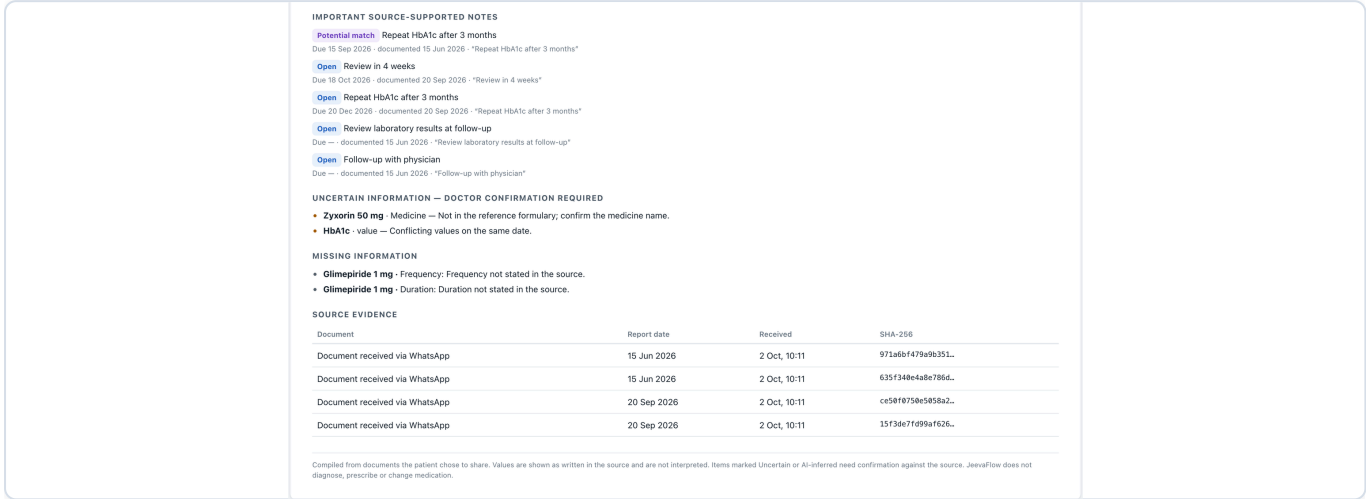
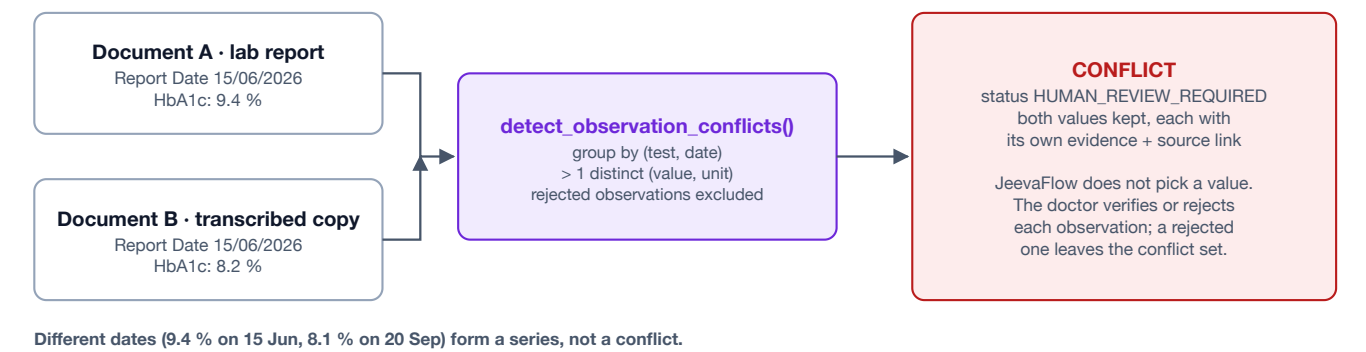


Figure 9.3 — Source-supported notes, uncertain and missing information, source evidence with SHA-256, and the standing notice.

The form carries a fixed notice: “Compiled from documents the patient chose to share. Values are shown as written in the source and are not interpreted. Items marked Uncertain or AI-inferred need confirmation against the source. JeevaFlow does not diagnose, prescribe or change medication.”

10 Conflict Detection

`detect_observation_conflicts()` (`conflicts.py`) groups non-rejected observations by (**test**, **date**) and reports a conflict when a group contains **more than one distinct (value, unit)**. The synthetic demo includes exactly this case:



Different dates (9.4 % on 15 Jun, 8.1 % on 20 Sep) form a series, not a conflict.

Figure 10.1 — Lab report and transcribed copy, both dated 15/06/2026, report HbA1c 9.4 % and 8.2 %.

STEP	BEHAVIOUR
Detect	Same test, same report date, different values → <code>type: CONFLICT</code>
Surface	<code>status: HUMAN_REVIEW_REQUIRED</code> , shown on the journey page, flagged in the form and listed under “Uncertain information”
Preserve	Both values are kept, each with its own evidence, document and source link
Review	The doctor verifies or rejects each observation; a rejected value leaves the conflict set
Never	JeevaFlow does not choose which value is correct

Values on **different** dates form a longitudinal series, not a conflict. Live result: one conflict (HbA1c, 15 Jun 2026: 9.4 % vs 8.2 %); the 8.1 % follow-up on 20 Sep 2026 appears as a later value (`test_conflict_detection_and_rejection`, `test_followup_on_later_date_is_not_a_conflict`).

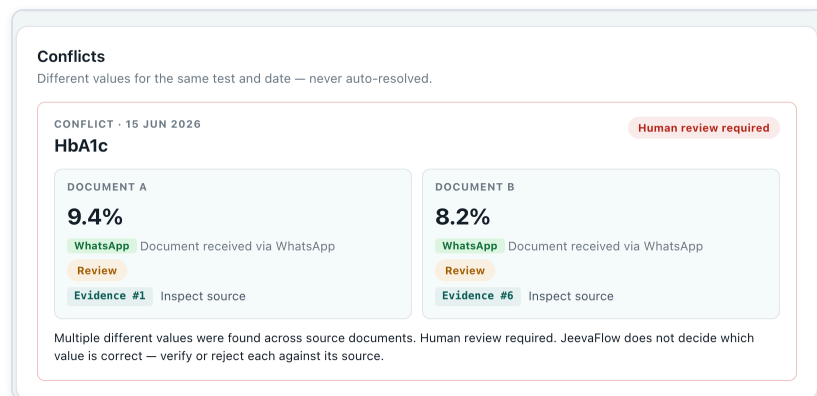
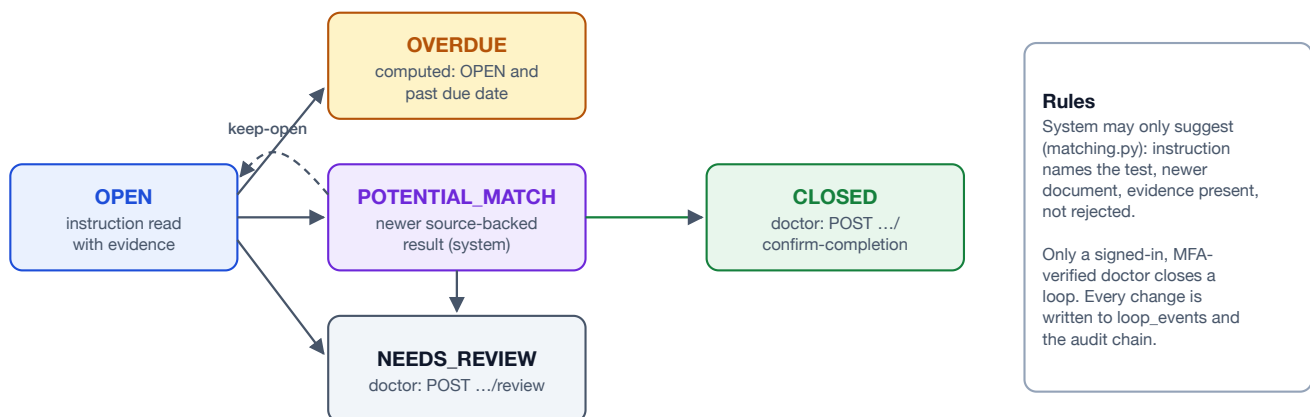


Figure 10.2 — Conflict panel in the doctor's journey view.

The doctor remains responsible for clinical interpretation.

11 Care Loops / Commitments

Every documented instruction (“Repeat HbA1c after 3 months”, “Review in 4 weeks”) becomes a **commitment** tracked as an **Open Loop** until a person confirms it was followed through.



Due dates are derived from the stated interval (“after 3 months”) from the report date; “Follow-up with physician” has no interval, so no due date.

Figure 11.1 — Loop states (`LoopState`) and who may cause each transition.

STATE	MEANING
OPEN	Instruction documented; nothing newer addresses it
OVERDUE	Computed: <code>OPEN</code> and past its due date
POTENTIAL_MATCH	A newer, source-backed result may fulfil it. Awaiting a person
NEEDS_REVIEW	A doctor flagged it (<code>POST .../commitments/{id}/review</code>)
CLOSED	A doctor confirmed completion (<code>POST .../commitments/{id}/confirm-completion</code>)

Matching rules (`matching.py`), all must hold:

1. The loop is active.
2. The instruction names the test.
3. The result comes from a different, newer document.
4. The result has evidence and is not rejected.

Doctor interaction:

- **Confirm completion:** closes the loop; recorded under the MFA-verified doctor, not a typed name. It requires a potential match (`test_confirmation_requires_potential_match`).
- **Keep open:** dismisses pending matches and reopens the loop.
- **Review:** marks the loop `NEEDS_REVIEW` .
- Rejecting the observation behind a match **withdraws** the match (`test_rejecting_supporting_observation_withdraws_match`).

Every transition is written to `loop_events` and the audit chain (`LOOP_DECISION`).

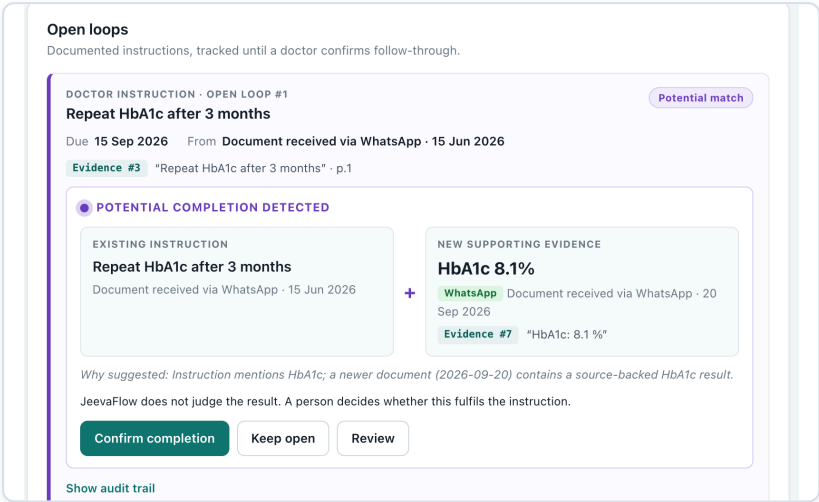


Figure 11.2 — Live: “Repeat HbA1c after 3 months” (15 Jun 2026, due 15 Sep 2026) has a potential match from the 20 Sep 2026 HbA1c 8.1 %. JeevaFlow states it does not judge the result; a person decides.

This prevents follow-up instructions from being buried inside scattered documents: they become visible, dated items with an owner and an outcome. In the live run the doctor saw five loops (one `POTENTIAL_MATCH` , four `OPEN`).

12 Doctor Authentication and MFA

Staff (doctors, auditors, admins) sign in with **two factors**: something they know and something they have.

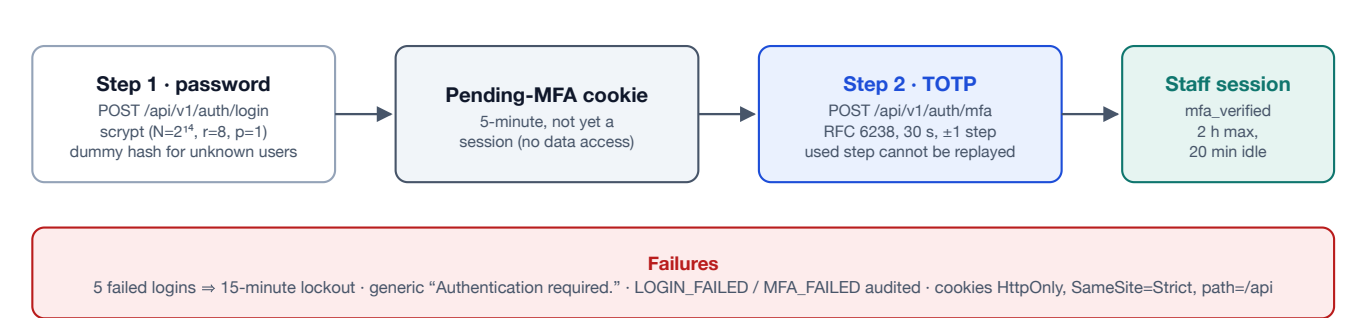


Figure 12.1 — Password + TOTP → authenticated staff session.

STEP	IMPLEMENTATION
1. Password	<code>POST /api/v1/auth/login</code> ; script (N = 2 ¹⁴ , r = 8, p = 1, 16-byte salt); unknown usernames cost the same time (dummy hash); success yields only a 5-minute <i>pending-MFA</i> cookie with no data access
2. TOTP	<code>POST /api/v1/auth/mfa</code> ; RFC 6238, 6 digits, 30-second step, ±1 step tolerance; a used step is stored and cannot be replayed
Session	Server-side; random token in an HttpOnly, SameSite=Strict cookie; only its HMAC stored; <code>mfa_verified</code> required by every staff route; 2-hour maximum, 20-minute idle timeout
Failures	Generic errors; 5 failed logins → 15-minute lockout; <code>LOGIN_FAILED</code> , <code>MFA_FAILED</code> , <code>LOGIN_LOCKED</code> audited

STEP	IMPLEMENTATION
CSRF	<code>X-CSRF-Token</code> required on every state-changing staff request; foreign <code>Origin</code> rejected

Why MFA matters here: staff accounts are the only path to clinical data. A phished or reused password alone is not enough to sign in (`test_password_alone_is_not_enough`). Replaying an intercepted TOTP code within its window is also refused (`test_totp_code_cannot_be_replayed`). This was observed directly during this audit: a second sign-in with the same 30-second code returned 401.

Demonstrated: `PASS dr.example: password + TOTP MFA` and `PASS auditor: password + TOTP MFA` in the live smoke test. The **auditor** uses the identical two-step flow; auditor and admin sessions are subject to the same idle timeout, lockout and CSRF rules.

STAFF SIGN-IN · STEP 2 OF 2

Enter your authenticator code

6-DIGIT CODE (TOTP)

Synthetic demo accounts: `dr.example`, `dr.other`, `auditor`, `admin`.
The password is `JEEVAFLOW_DEMO_STAFF_PASSWORD` in `backend/.env`. Get the current code with `venv/bin/python -m app.security.demo_totp`.

Figure 12.2 — Step 2 of staff sign-in. Demo TOTP codes come from a local command; no seed is shown in the UI.

13 Role-Based Access

Roles are defined in `identity.py` (`Role`) and enforced by `require_roles()` in `security/auth.py`: **deny by default**, every route declares the roles it accepts, and a wrong role produces a generic 403 plus a `DOCTOR_ACCESS_DENIED` audit with reason `ROLE`.

CAPABILITY	PATIENT	DOCTOR	AUDITOR	ADMIN
Authenticate	OTP (portal)	password + TOTP	password + TOTP	password + TOTP
Own consents, own documents (<code>/portal/*</code>)	✓	—	—	—
Consented patients' form, journey, loops, conflicts, evidence	—	✓ (consent + scope)	✗ 403	✗ 403
Verify / reject observations and facts; loop decisions	—	✓ (consent + scope)	✗	✗
Security status (<code>/security/status</code>)	—	✓	✓	✓
Audit events and chain verification (<code>/audit/*</code>)	—	✗ 403	✓	✓
Tamper / restore demo (<code>/demo/audit/*</code> , demo mode)	—	✗	✓	✓
Enrol patients, WhatsApp status/messages, run retention (<code>/admin/*</code>)	—	✗	✗ 403	✓
Any “list all patients” endpoint	does not exist	does not exist	does not exist	does not exist

Verified live (2 Oct 2026):

REQUEST	RESPONSE
Auditor → doctor form	403
Admin → doctor form	403
Doctor → audit events	403
Auditor → retention run	403
Admin → audit events	200

REQUEST	RESPONSE
Admin → retention run	200

`SERVICE` and `SYSTEM` appear only as audit actor types for internal components (webhook, worker, retention); they are not login roles.

Least privilege in practice:

- Admins operate the platform but **cannot read medical content**.
- Auditors see codes and references, not clinical data.
- Doctors see only what each patient shared with them.

Tests: `test_role_separation`, `test_missing_authentication`, `test_no_list_all_patients_or_phone_lookup`.

14 Consent-Based Authorization

Authentication answers “Who are you?”. In JeevaFlow that means a password + TOTP staff session.

Authorization answers “What are you allowed to see?”. In JeevaFlow that means role + active consent + scope + resource.

A fully authenticated doctor still sees **nothing** without patient consent. `authorize_doctor()` runs on every patient-data request:

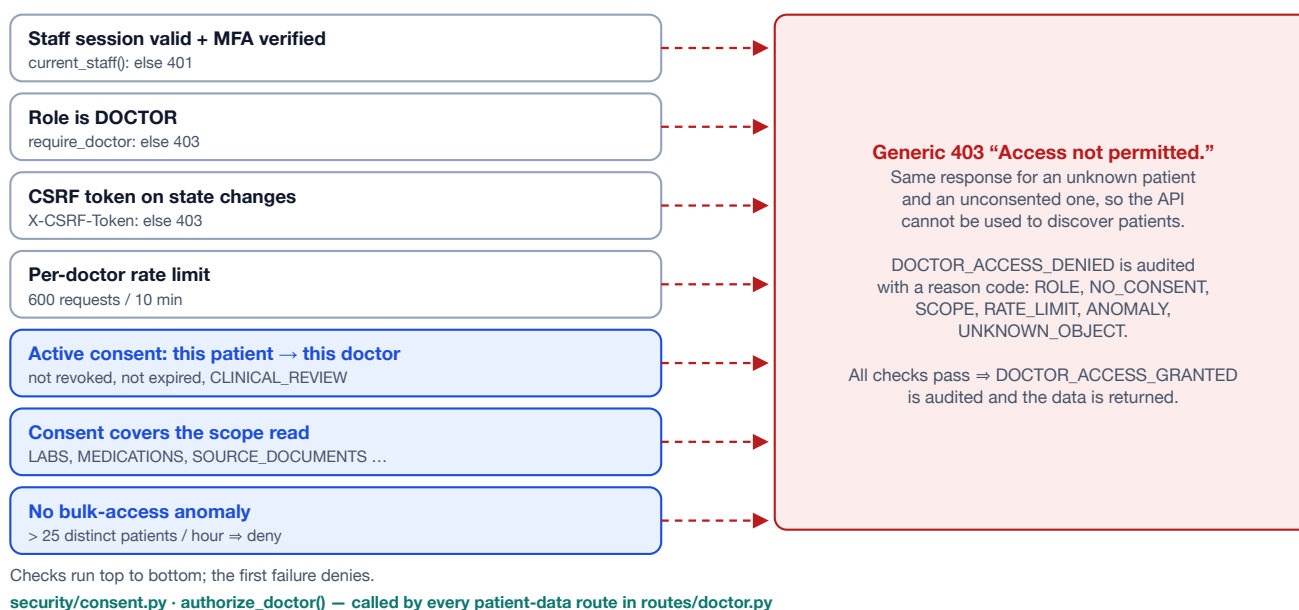


Figure 14.1 — Decision sequence: doctor identity + active patient consent + scope + requested resource → access.

Object-level routes (observation, fact, commitment, evidence) first resolve the object’s patient, then run the same check with the scope the object belongs to. Evidence additionally requires that the evidence is linked to a scope the consent covers. An unknown ID and an unconsented ID return the **same** 403.

SCENARIO	IDENTITY	CONSENT	RESULT
Dr. Example, patient consented to Dr. Example	MFA ✓	active ✓	Patient visible; data within scopes
Dr. Other, same patient	MFA ✓	none ✗	Not listed; direct request 403
Dr. Example after revocation	MFA ✓	revoked ✗	403 on the next request
Dr. Example, consent without <code>LABS</code>	MFA ✓	active, scope ✗	Lab values withheld; <code>/conflicts</code> 403

A bulk-access tripwire denies a doctor who touches more than 25 distinct patients within an hour (`ANOMALY_DETECTED` ; `test_bulk_access_anomaly`).

15

Evidence Security

Original documents are **never downloadable**. A doctor sees one evidence region at a time, rendered as a watermarked image through a short-lived, single-use token.

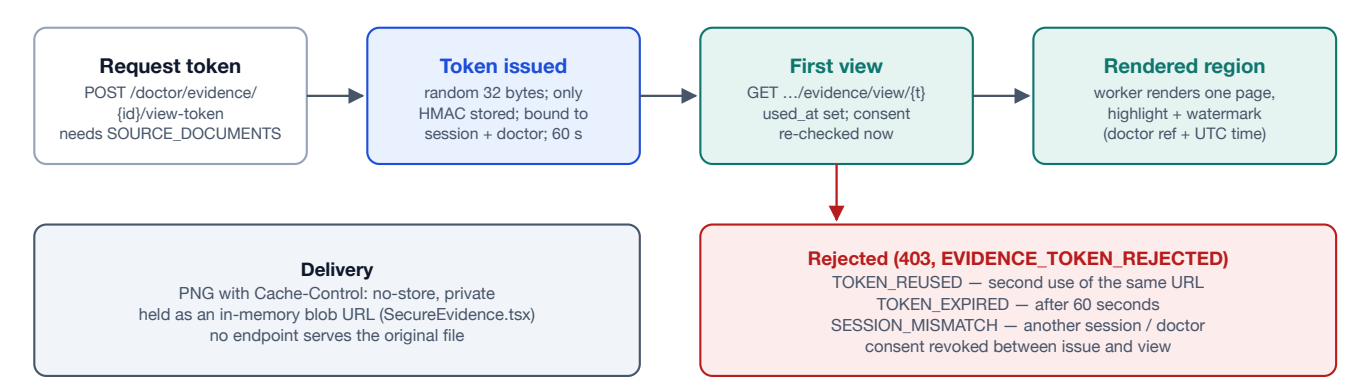


Figure 15.1 — Evidence token lifecycle (`routes/doctor.py`).

CONTROL	IMPLEMENTATION
Endpoints	<code>GET /doctor/evidence/{id}</code> (quote and linked items); <code>POST /doctor/evidence/{id}/view-token</code> ; <code>GET /doctor/evidence/view/{token}</code>
Scope	Viewing the source region requires the <code>SOURCE_DOCUMENTS</code> scope
Token	32 random bytes; only its HMAC is stored; bound to the doctor and the session; 60-second expiry; rate-limited (60 per 10 min)
Single use	<code>used_at</code> is set on first view; a second request → 403 <code>TOKEN_REUSED</code>
Live consent	Consent is re-checked at view time , not only at issue time
Rendering	The isolated worker decrypts in memory, renders only the referenced page, highlights the evidence box and burns in a watermark (doctor reference + UTC time + “JeevaFlow confidential – do not copy”)
Delivery	PNG with <code>Cache-Control: no-store, private</code> ; held in the browser as an in-memory blob URL (<code>SecureEvidence.tsx</code>)
Audit	<code>EVIDENCE_TOKEN_ISSUED</code> , <code>DOCUMENT_VIEWED</code> , <code>EVIDENCE_TOKEN_REJECTED</code> (with reason)

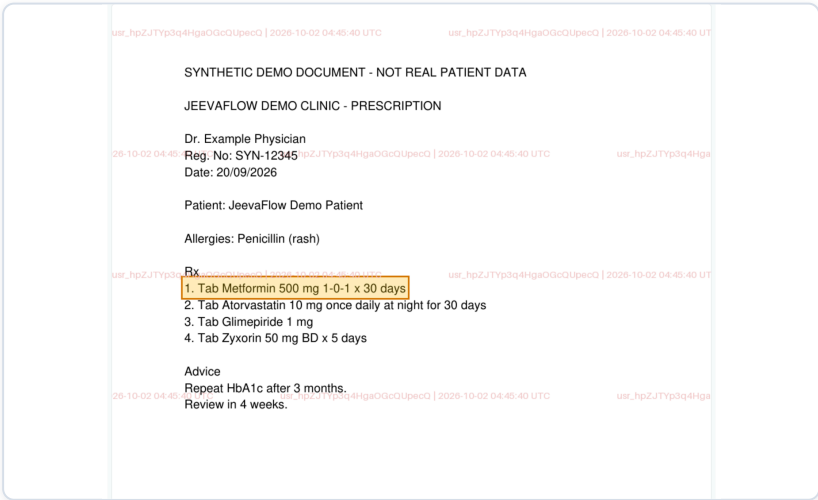
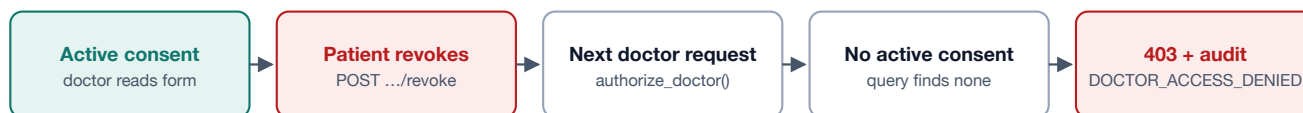


Figure 15.2 — Live watermarked evidence render: the Metformin line is highlighted; the doctor’s opaque user reference and timestamp are tiled across the page.

Demonstrated: `PASS` watermarked evidence view, `PASS` evidence token is single use. A privileged URL that is copied into a chat, a log or a browser history is worthless once used or after 60 seconds, and it cannot be opened from another session. The watermark discourages screenshots and makes any leaked image attributable. It cannot *prevent* someone photographing a screen.

16 Access Revocation



No caches or long-lived grants: the patient list, form, journey, loops, conflicts, evidence and view tokens all re-run the consent query per request, so revocation takes effect on the very next call (live check: 403 immediately after revoke).

Figure 16.1 — ACTIVE CONSENT → doctor access → patient revokes → authorization check fails → access denied.

Access denied immediately after revocation. Revocation is enforced at *authorization time* on the server. It is not a UI change and not a cached flag. The consent query in `authorize_doctor()` runs on every request, so the doctor's very next call fails.

The doctor's patient list is computed from active consents on every request, so a revoked patient also disappears from "My patients". Evidence tokens issued *before* revocation also fail, because consent is re-checked at view time (`test_evidence_view_requires_source_scope_and_live_consent`).

Live result (2 Oct 2026):

1. Before revocation, Dr. Example's form request returned `200`.
2. The patient called `POST /portal/consents/{ref}/revoke`.
3. The next form request returned `403`, and the patient list was empty.

The patient portal offers two controls per active consent:

- **Revoke:** permanently withdraws the consent.
- **End now:** sets the expiry to the current time.

Both are audited.

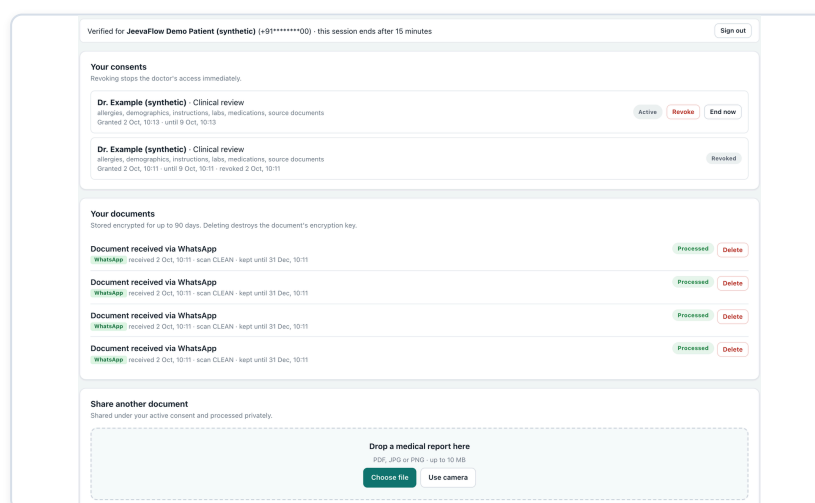


Figure 16.2 — Patient portal: active and revoked consents with Revoke / End now; documents with scan status, retention date and Delete (crypto-shred).

17 Audit Trail

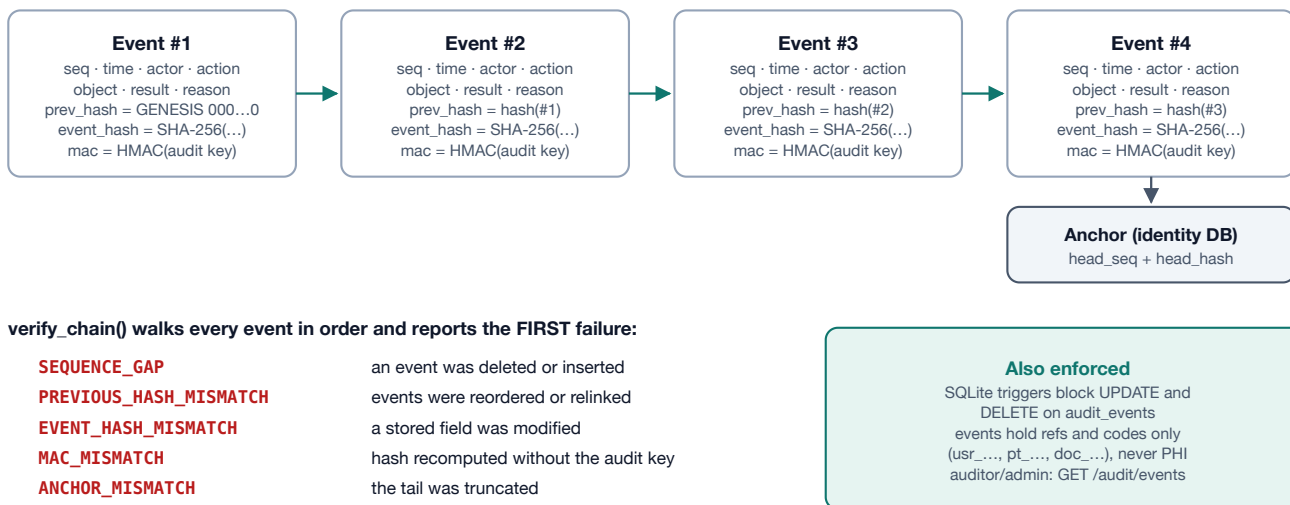


Figure 17.1 — USER ACTION → AUDIT EVENT → TAMPER-EVIDENT CHAIN → AUDITOR VERIFICATION.

17.1 How the chain works (security/audit.py)

- Each event records the sequence number, timestamp, actor type and reference, action, object type and reference, result, reason, and the **previous event's hash**.
- `event_hash = SHA-256(canonical JSON of those fields)` and `mac = HMAC-SHA256(audit key, event_hash)`.
- The latest `(head_seq, head_hash)` is written to an **anchor** in the separate identity database.
- SQLite triggers make `audit_events` **append-only**: UPDATE and DELETE are refused (`test_audit_table_is_append_only`).
- Events contain **only references and codes**. Values are validated against a safe pattern and replaced with `REDACTED` otherwise, so no PHI enters the log (`test_audit_events_contain_no_phi`).

An attacker who edits a row breaks its hash. Recomputing the hashes needs the audit key for the MAC. Deleting the newest events is caught by the anchor (`test_audit_detects_deleted_tail`).

17.2 Coverage

The audit trail defines **50 action types**, including:

- **Channel:** webhook signature, replay and rate-limit events.
- **Ingestion:** document received, validated, rejected, scanned, encrypted and deleted.
- **Patient trust:** OTP issued, verified, failed and locked; consent granted, revoked and expired.
- **Staff:** logins, MFA, logout and session expiry.
- **Clinical access:** doctor access granted and denied, evidence tokens, document views, and fact, observation and loop decisions.
- **Administration:** enrolment and audit-chain verification.

In the live verification instance the chain held 166 events (several full demo cycles) when the tamper test below was run.

17.3 Auditor verification

The auditor (or admin) has two endpoints:

- `GET /api/v1/audit/events` lists recent events.
- `POST /api/v1/audit/verify` walks the whole chain and returns `valid`, the number of events, the head hash, and on failure `broken_at_seq` with a reason.

Each verification is itself audited (`AUDIT_CHAIN_VERIFIED` / `AUDIT_CHAIN_BROKEN`). **Demonstrated:** `PASS` `audit chain verified` .

18

Tamper Demonstration

`POST /api/v1/demo/audit/tamper` and `POST /api/v1/demo/audit/restore` simulate an attacker with **direct database write access**. Both require an auditor or admin session and **demo mode** (404 otherwise).

- **Tamper:** temporarily drops the append-only trigger, flips one event's `result` field (e.g. `SUCCESS` ↔ `DENIED`), remembers the original in memory, and re-creates the trigger.
- **Restore:** writes the original values back.

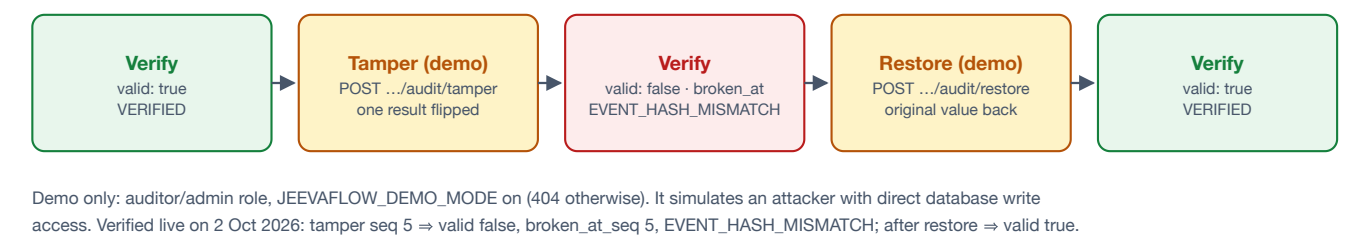


Figure 18.1 — Normal chain → PASS; tampered event → FAIL at that event; restored chain → PASS.

Live API result (2 Oct 2026):

STEP	RESPONSE
Verify	<code>valid: true</code> , 166 events
Tamper <code>seq 5</code>	<code>tampered_seq: 5</code>
Verify	<code>valid: false</code> , <code>broken_at_seq: 5</code> , <code>reason: EVENT_HASH_MISMATCH</code>
Restore	<code>restored: 1</code>
Verify	<code>valid: true</code> (the <code>AUDIT_CHAIN_BROKEN</code> event itself stays in the chain)

Tamper-evident audit trail						
Each event stores the SHA-256 hash of the previous event, plus an HMAC. Changing any record breaks the chain.						
Audit chain BROKEN at event #200 (EVENT_HASH_MISMATCH). The record was modified after it was written.						
#	Time (UTC)	Actor	Action	Object	Result	Hash
204	2026-10-02 04:44:48	STAFF usr_qTtacsLvLBRQ_xKdNC5Sw	AUDIT_CHAIN_BROKEN EVENT_HASH_MISMATCH	—	FAILED	5afa5b587a...
203	2026-10-02 04:44:44	STAFF usr_qTtacsLvLBRQ_xKdNC5Sw	AUDIT_CHAIN_VERIFIED	—	SUCCESS	285313a72b...
202	2026-10-02 04:44:43	STAFF usr_qTtacsLvLBRQ_xKdNC5Sw	MFA_VERIFIED	—	SUCCESS	39882c3a8...
201	2026-10-02 04:44:43	STAFF usr_qTtacsLvLBRQ_xKdNC5Sw	LOGIN_PASSWORD_OK	—	SUCCESS	cbf89642c1...
200	2026-10-02 04:44:41	STAFF usr_IDEkcvLxAsaTpKz37p8Cg	MFA_VERIFIED	—	DENIED	b47d38b484...
199	2026-10-02 04:44:41	STAFF usr_IDEkcvLxAsaTpKz37p8Cg	LOGIN_PASSWORD_OK	—	SUCCESS	823ae4fc6b...
198	2026-10-02 04:44:35	STAFF usr_hp22Ty3q4HgaDc0BsecQ	DOCTOR_ACCESS_GRANTED DOCTOR_FORM	pt_9ryxN5thA802_wlqKscrfg	SUCCESS	23bd6d0646...

Figure 18.2 — Security page after tampering with event #200: “Audit chain BROKEN at event #200 (EVENT_HASH_MISMATCH)”, with the modified row highlighted.

This is a **security demonstration**, not a production feature. In production these endpoints return 404 because demo mode cannot be enabled there.

19

Data Retention

Retention is implemented in `retention.py` . `run_retention()` runs at server start and **every 300 seconds**; an admin can also trigger it with `POST /api/v1/admin/retention/run` .

RULE	BEHAVIOUR
Unverified uploads	Quarantined documents whose transaction is <code>EXPIRED</code> or <code>LOCKED</code> are crypto-shredded (reason <code>UNVERIFIED_EXPIRED</code>)
Retention period	Every document gets <code>retain_until = received + 90 days (JEEVAFLOW_DOCUMENT_RETENTION_DAYS)</code> ; past that it is crypto-shredded (reason <code>RETENTION</code> , audited as <code>DOCUMENT_RETENTION_EXPIRED</code>)
Consents	Consents past expiry are audited once as <code>CONSENT_EXPIRED</code>
Tokens and sessions	Expired evidence tokens (after 5 minutes) and expired sessions (after 1 hour) are deleted
Patient deletion	<code>DELETE /api/v1/portal/documents/{ref}</code> crypto-shreds a document on request
Demo reset	<code>POST /api/v1/demo/reset</code> crypto-shreds all synthetic demo documents (demo mode, localhost only)

Deletion order:

1. Destroy the wrapped data key.
2. Overwrite and unlink the encrypted object.
3. Delete the dependent rows (evidence, observations, commitments, facts, matches).
4. Write the audit event.

The key is destroyed **first** because overwriting files is not guaranteed on SSDs or copy-on-write filesystems. Once the key is gone, any leftover ciphertext is unreadable.

Live admin run on a fresh instance: `{transactions_expired: 0, unverified_documents_deleted: 0, retention_deleted: 0, consents_expired: 0}`. Nothing was due.

Why it matters: limiting how long data exists limits what a future breach, subpoena or misconfiguration can expose. Unverified uploads (for example, media sent from a phone that is not the patient's) disappear within the transaction lifetime.

The consent record also stores a `retention_days` value (90). Document expiry itself uses the global setting, so a per-consent retention period is not enforced separately.

20 Security Status

`GET /api/v1/security/status` (any MFA-verified staff role) returns a **live, self-describing security posture**:

- environment;
- demo mode;
- generation time;
- the current audit-chain verification result;
- **16 controls**, each with a status, a detail line, a grade (`IMPLEMENTED` , `DEMO IMPLEMENTATION` or `PRODUCTION REQUIRED`) and the named production replacement.

CONTROL	LIVE STATUS	GRADE
Encryption	ACTIVE	IMPLEMENTED
Key management	ACTIVE	DEMO IMPLEMENTATION (KEK from environment)
OTP Protection	ACTIVE	IMPLEMENTED
Consent	ACTIVE / ENFORCED	IMPLEMENTED
Doctor MFA	ACTIVE	IMPLEMENTED
File Scan	PASSED	DEMO IMPLEMENTATION (heuristic engine; ClamAV not installed)
OCR	PRIVATE	DEMO IMPLEMENTATION (process isolation, not container)
External AI	DISABLED	IMPLEMENTED
PHI Logging	BLOCKED	IMPLEMENTED
Audit Chain	VERIFIED	IMPLEMENTED
Document Access	AUTHORIZED	IMPLEMENTED
Document Retention	ACTIVE	IMPLEMENTED

CONTROL	LIVE STATUS	GRADE
WhatsApp Channel	ENFORCED	DEMO IMPLEMENTATION (no restricted Twilio API key configured)
Rate limiting	ACTIVE	DEMO IMPLEMENTATION (in-memory)
Transport	LOCAL HTTP	DEMO IMPLEMENTATION
Database encryption at rest	FIELD-LEVEL	PRODUCTION REQUIRED

The endpoint’s own statement is deliberately modest (“Defense-in-depth security architecture...”), and the page footer states that these technical controls “do not by themselves establish regulatory compliance” (`test_security_dashboard_is_honest`).

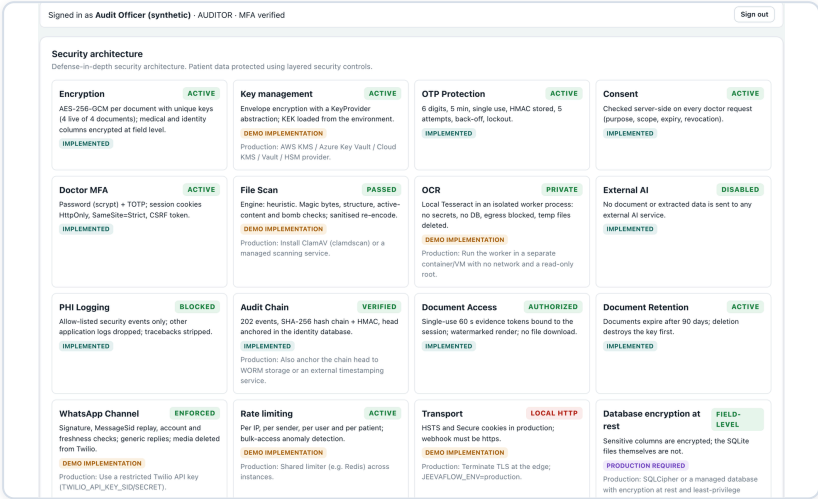


Figure 20.1 — Security page (auditor session) showing control grades and named production replacements.

21

Frontend

The frontend (`frontend/` , React 19 + TypeScript + Vite, plain CSS) has four sections reachable from the top navigation. The Vite dev server proxies `/api` to the backend so the browser talks to a single origin. Every page shows the principle banner and a footer stating that data is synthetic and that JeevaFlow does not diagnose, prescribe or change medication. All screenshots in this report are from a live run against an isolated synthetic instance on 2 October 2026.

SCREEN	PURPOSE	KEY COMPONENTS
Doctor console (<code>/</code>)	Staff sign-in (password → TOTP); “My patients” (consented only); tabs <i>Doctor-ready form</i> and <i>Journey & review</i>	<code>DoctorConsole.tsx</code> , <code>StaffLogin.tsx</code> , <code>DoctorForm.tsx</code> , <code>Timeline.tsx</code> , <code>OpenLoops.tsx</code> , <code>Conflicts.tsx</code> , <code>SecureEvidence.tsx</code> , <code>EvidenceViewer.tsx</code>
Patient portal (<code>/portal</code>)	OTP entry; consents (grant, Revoke, End now); documents with status, scan result and retention date; upload or camera capture	<code>PatientPortal.tsx</code> , <code>DocumentUpload.tsx</code> , <code>CameraCapture.tsx</code> , <code>DocumentsList.tsx</code>
Security (<code>/security</code>)	Control grid from <code>/security/status</code> ; tamper-evident audit trail with Verify, Tamper and Restore (demo)	<code>SecurityDashboard.tsx</code>
Demo (<code>/demo</code>)	Synthetic WhatsApp phone (four documents, “Hi”, forged signature); webhook result; demo accounts; Reset demo	<code>DemoConsole.tsx</code> , <code>StoryStrip.tsx</code>

Frontend security details:

- CSRF tokens are kept in memory only, never in `localStorage` .
- Evidence images exist only as in-memory blob URLs.
- Upload progress shows the backend’s real pipeline stages.

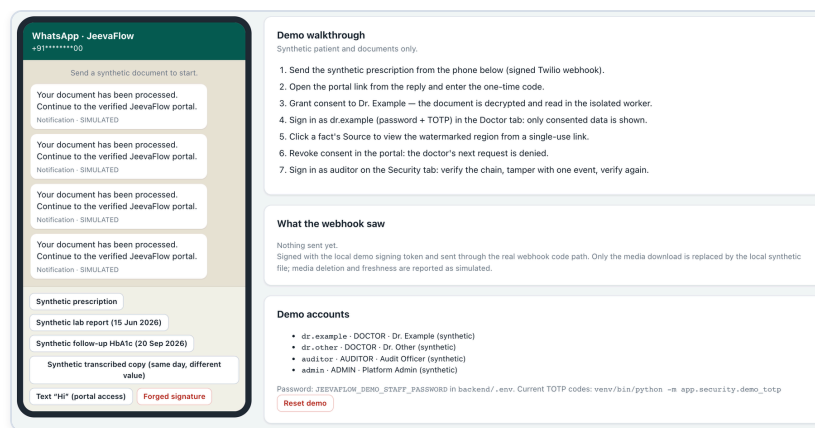


Figure 21.1 — Demo control room: synthetic phone with the four synthetic documents, the generic replies (notifications marked *SIMULATED*) and the seeded synthetic accounts.

22 Backend and Verified Technology Stack

The backend is a single FastAPI application (`app/main.py`, version 2.0.0) with seven routers, served by Uvicorn. It loads configuration from the environment and refuses to start without its secrets. A background task runs the retention job every five minutes.

API hardening in `main.py`:

- **Origin check:** state-changing requests from a foreign `Origin` are rejected.
- **Request-size cap:** oversized requests are refused before they are read.
- **Security headers:**
 - `nosniff`
 - `X-Frame-Options: DENY`
 - `Referrer-Policy: no-referrer`
 - `Cross-Origin-Resource-Policy: same-origin`
 - `Permissions-Policy` (camera, microphone and geolocation disabled)
 - CSP `default-src 'none'` and `no-store` on API responses
 - HSTS in production
- **Strict CORS:** only the configured origins.
- **Generic errors:** generic 500s and validation errors that do not echo input.
- **Upload handling:** uploads are kept in memory, never spooled to disk.

Verified Technology Stack

TECHNOLOGY	PURPOSE	WHERE USED	VERIFIED?
Python 3 (venv: 3.14)	Backend language	<code>backend/</code>	✓ interpreter in <code>venv</code>
FastAPI 0.142	REST API framework	<code>app/main.py</code> , <code>app/routes/*</code>	✓ <code>requirements.txt</code> , installed
Uvicorn	ASGI server	<code>python -m app</code> , <code>run_demo.sh</code>	✓
Pydantic / pydantic-settings	Request validation	route models	✓
SQLAlchemy + SQLite	ORM; <code>jeevaflow.db</code> and separate <code>identity.db</code>	<code>database.py</code> , <code>identity.py</code> , <code>models.py</code>	✓
cryptography (AESGCM, HKDF)	AES-256-GCM documents and fields; key derivation	<code>security/crypto.py</code>	✓
hashlib script, hmac (stdlib)	Password hashing; TOTP (RFC 6238, own implementation); OTP/session HMACs	<code>security/auth.py</code> , <code>security/otp.py</code>	✓

TECHNOLOGY	PURPOSE	WHERE USED	VERIFIED?
PyMuPDF 1.28	PDF parsing, text layer, rendering, sanitisation, structural active-content scan	<code>worker.py</code> , <code>security/pdf_active_content.py</code>	✓
Tesseract 5.5.3 via pytesseract	Local OCR	<code>worker.py</code> , <code>ocr.py</code>	✓ binary installed
Pillow	Image decode, re-encode, watermark rendering	<code>worker.py</code>	✓
OpenCV + NumPy	Image quality gate (sharpness, brightness)	<code>quality.py</code>	✓
ClamAV	Signature malware scan if installed	<code>security/scanner.py</code>	✗ not installed here (heuristic engine used)
httpx	Twilio REST (media download/delete, notifications, freshness)	<code>whatsapp.py</code>	✓ code; live Twilio not exercised in this audit
Twilio WhatsApp	Messaging transport	<code>whatsapp.py</code> , <code>routes/webhook.py</code>	🕒 signature path verified via simulator; credentials not configured
python-dotenv	<code>.env</code> loading	<code>config.py</code>	✓
pytest	Backend tests	<code>backend/tests/</code>	✓ 156 passed
React 19 + TypeScript + Vite 8	Frontend	<code>frontend/</code>	✓ <code>package.json</code>
Vitest + Testing Library	Frontend tests	<code>frontend/src/test/</code>	✓ 8 passed
Generative AI / LLM	—	—	Not used (no AI SDK in requirements; dashboard: External AI DISABLED)

23 API Architecture

All routes below were read from the application's routers (`app/routes/*.py`) and match the OpenAPI listing (served at `/docs` and `/openapi.json` in development only).

Notation:

- **Staff** means an MFA-verified staff session.
- **Consent** means `authorize_doctor()` with the named scope.
- **CSRF** means the `X-CSRF-Token` header, required on every state-changing session request.
- **Demo-local** means demo mode, a loopback client, a local `Host` header and no proxy headers.

GROUP	METHOD	ENDPOINT	PURPOSE	SECURITY REQUIREMENT	ROLE
Health	GET	<code>/</code>	Service name, version	Public	Any
Health	GET	<code>/api/v1/health</code>	Liveness	Public	Any
WhatsApp	POST	<code>/api/v1/webhooks/whatsapp</code>	Inbound messages and media	Twilio signature (fails closed), replay, account, freshness, rate limits	Twilio
Patient portal	POST	<code>/api/v1/portal/verify</code>	Verify OTP, start session	Transaction + OTP; IP rate limit	Patient
Patient portal	GET	<code>/api/v1/portal/session</code>	Patient overview, options	Patient session	Patient
Patient portal	POST	<code>/api/v1/portal/logout</code>	End session	Patient session + CSRF	Patient
Consent	POST	<code>/api/v1/portal/consents</code>	Grant consent (and process pending documents)	Patient session + CSRF	Patient
Consent	POST	<code>/api/v1/portal/consents/{consent_ref}/apply</code>	Share pending documents under an existing consent	Patient session + CSRF; consent active	Patient
Consent	POST	<code>/api/v1/portal/consents/{consent_ref}/revoke</code>	Revoke	Patient session + CSRF; own consent	Patient
Consent	POST	<code>/api/v1/portal/consents/{consent_ref}/expire</code>	End now	Patient session + CSRF; own consent	Patient

GROUP	METHOD	ENDPOINT	PURPOSE	SECURITY REQUIREMENT	ROLE
Documents	POST	/api/v1/portal/documents	Upload (validate → scan → encrypt → process)	Patient session + CSRF + active consent; upload rate limit	Patient
Documents	GET	/api/v1/portal/documents	List own documents	Patient session	Patient
Documents	DELETE	/api/v1/portal/documents/{document_ref}	Crypto-shred own document	Patient session + CSRF	Patient
Authentication	POST	/api/v1/auth/login	Password step	Lockout, rate limits, generic errors	Staff
Authentication	POST	/api/v1/auth/mfa	TOTP step	Pending-MFA cookie; replay block	Staff
Authentication	GET	/api/v1/auth/me	Current staff user + CSRF token	Staff	Doctor / Auditor / Admin
Authentication	POST	/api/v1/auth/logout	End session	Staff + CSRF	Doctor / Auditor / Admin
Doctor	GET	/api/v1/doctor/patients	Patients with active consent to me	Staff	Doctor
Doctor	GET	/api/v1/doctor/patients/{patient_ref}/form	Doctor-ready form	Consent (sections by scope)	Doctor
Doctor	GET	/api/v1/doctor/patients/{patient_ref}/journey	Timeline, loops, conflicts, documents	Consent (sections by scope)	Doctor
Doctor	GET	/api/v1/doctor/patients/{patient_ref}/loops	Open Loops	Consent: INSTRUCTIONS	Doctor
Doctor	GET	/api/v1/doctor/patients/{patient_ref}/conflicts	Conflicts	Consent: LABS	Doctor
Doctor	PATCH	/api/v1/doctor/observations/{observation_id}/verify	Verify observation	Consent: LABS + CSRF; evidence required	Doctor
Doctor	PATCH	/api/v1/doctor/observations/{observation_id}/reject	Reject (withdraws matches)	Consent: LABS + CSRF	Doctor
Doctor	PATCH	/api/v1/doctor/facts/{fact_ref}/confirm	Confirm medication/allergy fact	Consent: category scope + CSRF	Doctor
Doctor	PATCH	/api/v1/doctor/facts/{fact_ref}/reject	Reject fact	Consent: category scope + CSRF	Doctor
Doctor	GET	/api/v1/doctor/commitments/{commitment_id}	Loop detail + history	Consent: INSTRUCTIONS	Doctor
Doctor	POST	/api/v1/doctor/commitments/{commitment_id}/confirm-completion	Close loop	Consent: INSTRUCTIONS + CSRF; potential match	Doctor
Doctor	POST	/api/v1/doctor/commitments/{commitment_id}/keep-open	Dismiss matches	Consent: INSTRUCTIONS + CSRF	Doctor
Doctor	POST	/api/v1/doctor/commitments/{commitment_id}/review	Mark needs review	Consent: INSTRUCTIONS + CSRF	Doctor
Evidence	GET	/api/v1/doctor/evidence/{evidence_id}	Quote, location, linked items	Consent + evidence scope	Doctor
Evidence	POST	/api/v1/doctor/evidence/{evidence_id}/view-token	Issue 60 s single-use token	Consent: SOURCE_DOCUMENTS + CSRF; rate limit	Doctor
Evidence	GET	/api/v1/doctor/evidence/view/{token}	Watermarked PNG of the region	Same session + doctor; unused; unexpired; consent re-checked	Doctor
Security	GET	/api/v1/security/status	Live control status	Staff	Doctor / Auditor / Admin
Audit	GET	/api/v1/audit/events	Recent audit events	Staff	Auditor / Admin
Audit	POST	/api/v1/audit/verify	Verify chain	Staff + CSRF	Auditor / Admin
Demo	POST	/api/v1/demo/audit/tamper	Simulate DB tampering	Staff + CSRF + demo mode	Auditor / Admin
Demo	POST	/api/v1/demo/audit/restore	Undo simulated tampering	Staff + CSRF + demo mode	Auditor / Admin
Admin	POST	/api/v1/admin/patients	Enrol patient (generic error)	Staff + CSRF	Admin
Admin	GET	/api/v1/admin/whatsapp/status	Credential-free channel status	Staff	Admin

GROUP	METHOD	ENDPOINT	PURPOSE	SECURITY REQUIREMENT	ROLE
Admin	GET	<code>/api/v1/admin/whatsapp/messages</code>	Message log (masked senders)	Staff	Admin
Admin	POST	<code>/api/v1/admin/retention/run</code>	Run retention now	Staff + CSRF	Admin
Demo	GET	<code>/api/v1/demo</code>	Demo state	Demo-local	Local demo
Demo	POST	<code>/api/v1/demo/whatsapp</code>	Signed simulated WhatsApp message (real webhook path)	Demo-local	Local demo
Demo	GET	<code>/api/v1/demo/phone</code>	Synthetic phone view	Demo-local	Local demo
Demo	POST	<code>/api/v1/demo/reset</code>	Crypto-shred demo documents	Demo-local	Local demo
Demo	GET	<code>/api/v1/demo/files/{name}</code>	Download a synthetic demo PDF	Demo-local	Local demo

24 Complete End-to-End Demo

A script judges can follow on a local machine with the demo running. It needs:

- the frontend at `http://localhost:5173`;
- demo mode enabled;
- **synthetic data only**.

The staff password is read from the local `.env`. TOTP codes come from `venv/bin/python -m app.security.demo_totp`. Neither is reproduced here.

STEP	ACTION (WHERE)	WHAT TO OBSERVE	BACKED BY
1	Demo → <i>Synthetic prescription</i>	The synthetic phone sends a document	POST <code>/demo/whatsapp</code>
2	(automatic)	“What the webhook saw”: signature VERIFIED ; <i>Forged signature</i> shows 403	<code>routes/webhook.py</code>
3	Phone	Generic reply with portal link, then the code in a separate message	REPLIES
4	Open the link → Patient portal	Enter the 6-digit code; session starts (15 min)	POST <code>/portal/verify</code>
5	Portal → grant consent to <i>Dr. Example</i> , all scopes, 7 days	Consent ACTIVE	POST <code>/portal/consents</code>
6	(automatic)	Quarantined document decrypted and processed in the isolated worker; phone shows “processed”	<code>process_document()</code>
7	(automatic)	Medications, allergy, prescriber, instructions extracted with states	<code>extraction.py</code> , <code>facts.py</code>
8	(automatic)	Every item stored with quote, page, bbox (quote-or-reject)	<code>provenance.py</code>
9	Doctor → sign in <code>dr.example</code> → TOTP	“MFA verified”	<code>/auth/login</code> , <code>/auth/mfa</code>
10	Doctor → My patients	Only the consenting patient is listed; sign in as <code>dr.other</code> to see an empty list	GET <code>/doctor/patients</code>
11	<i>Journey & review</i>	Timeline, open loops, conflicts, shared documents	<code>.../journey</code>
12	<i>Doctor-ready form</i>	Medications with SOURCE_FACT / AI-inferred / Uncertain / Missing; labs; allergies; notes	<code>.../form</code>
13	Review medications and observations	Confirm or Reject facts; Verify or Reject observations	PATCH <code>.../facts/...</code> , <code>.../observations/...</code>
14	Send the <i>lab report</i> and <i>transcribed copy</i> (repeat 3–5 or apply existing consent)	Conflict HbA1c 9.4 % vs 8.2 % on 15 Jun 2026, “Human review required”; add the <i>follow-up</i> for a potential match	<code>.../conflicts</code> , <code>.../loops</code>

STEP	ACTION (WHERE)	WHAT TO OBSERVE	BACKED BY
15	Form → Source → View source region	Highlighted, watermarked page render	<code>.../view-token</code> , <code>.../view/{token}</code>
16	(repeat the same token URL)	403 <code>TOKEN_REUSED</code>	single-use token
17	Patient portal → Revoke	Consent REVOKED	<code>POST .../revoke</code>
18	Doctor → Refresh / open form	Patient gone; 403 on direct request	<code>authorize_doctor()</code>
19	Security → sign in <code>auditor</code> → Verify audit chain → Tamper → Verify → Restore (demo) → Verify	VERIFIED → BROKEN at that event → VERIFIED	<code>/audit/verify</code> , <code>/demo/audit/*</code>

The same flow is automated by `backend/test_api.py` (steps 1–6, 9–10, 12, 15–19); `run_demo.sh` starts the server and runs it.

25 Security & Test Validation

156 automated backend tests passed in the current development environment (pytest, 2 October 2026). That is the 119-test suite documented in the README, plus 37 tests for the structural PDF active-content detector added on 2 October 2026. The frontend suite (Vitest) passed **8 of 8**. These results show the implemented controls behave as designed in this environment. They are **not** a certification, a penetration test or evidence of production readiness.

TEST FILE	TESTS	FOCUS
<code>tests/test_security.py</code>	60	OTP, consent, roles, MFA, CSRF, injection, files, encryption, evidence tokens, logging, audit chain, retention, worker isolation
<code>tests/test_pdf_active_content.py</code>	37	Structural PDF scanning: JavaScript, launch/remote actions, <code>/AA</code> , XFA, rich media, embedded files, C2PA manifests, malformed PDFs, pipeline and endpoint
<code>tests/test_whatsapp.py</code>	19	Signature, replay, account, freshness, unenrolled senders, media download/deletion, generic replies, rate limits
<code>tests/test_core.py</code>	17	Ingestion, provenance, OCR, image quality gate, extraction, idempotency
<code>tests/test_facts.py</code>	13	Medication/allergy/prescriber extraction and fact states
<code>tests/test_loops.py</code>	10	Open Loops, conflicts, potential matches, doctor form sections and scopes
Total (backend)	156	all passed

Live security smoke test (`backend/test_api.py`)

Re-run for this report on 2 October 2026 against an isolated instance (fresh random keys, temporary databases, synthetic patient). **13 of 13 checks passed.**

TEST	RESULT
Health	PASS
WhatsApp signature verification	PASS
Generic WhatsApp reply	PASS
OTP verification	PASS
Consent + document processing	PASS
Doctor password + TOTP MFA	PASS
Consent-based patient visibility (“doctor sees only the consented patient”)	PASS
Doctor-ready form (with medications)	PASS
Watermarked evidence	PASS
Single-use evidence token	PASS
Immediate access revocation	PASS
Auditor MFA	PASS

TEST	RESULT
Audit chain verification	PASS

Additional live checks performed for this report:

CHECK	RESULT
Dr. Other with no consent	Empty list; form request 403
Conflict detection	HbA1c 9.4 % vs 8.2 % on 15 Jun 2026: <code>HUMAN_REVIEW_REQUIRED</code>
Potential match	20 Sep 2026 HbA1c result matched to the “Repeat HbA1c” loop
Tamper demo	Chain broken at seq 5 (<code>EVENT_HASH_MISMATCH</code>), then restored to valid
Role separation	403 / 200 exactly as in the matrix in Section 13
TOTP replay blocked	Reusing a 30-second code returned 401
Per-sender webhook rate limit	Triggered after repeated messages
Structural PDF scan	A static PDF carrying a C2PA manifest was accepted, and the manifest was stripped from the stored copy

26 Security Threat Model

Only defenses present in the code are listed. “Evidence” names the automated test(s) or the live check.

THREAT	JEEVAFLOW DEFENSE	EVIDENCE
Forged or replayed WhatsApp webhook	HMAC signature (fails closed), MessageSid replay table, AccountSid and freshness checks	<code>test_unsigned_and_wrongly_signed_requests_are_rejected</code> , <code>test_replayed_message_sid_is_ignored</code> , <code>test_stale_message_is_dropped</code> ; live PASS
Compromised or shared phone / chat history	Chat carries no medical content; data only via OTP-verified portal; quarantine until consent	<code>test_replies_contain_no_medical_information</code> , <code>test_media_is_quarantined_until_otp_and_consent</code>
Unknown sender pushing files	Generic reply, no download, no patient created files	<code>test_unenrolled_number_gets_generic_reply_and_no_download</code>
OTP guessing / reuse	5 attempts + back-off + lock, single use, 5-min expiry, HMAC storage	<code>test_otp_brute_force_locks_transaction</code> , <code>test_otp_is_single_use</code> , <code>test_otp_never_stored_in_plaintext</code>
Unauthorized doctor access	Password + TOTP MFA, then consent + scope on every request	<code>test_password_alone_is_not_enough</code> , <code>test_wrong_doctor_is_denied</code> ; live PASS
Stolen password	TOTP second factor, lockout, generic errors	<code>test_login_errors_are_generic_and_lock_out</code>
Intercepted TOTP code	Used time-step stored; replay refused	<code>test_totp_code_cannot_be_replayed</code> ; observed live
Session hijack / CSRF	HttpOnly SameSite=Strict cookies, HMAC-stored tokens, CSRF header, Origin check, idle timeout	<code>test_csrf_token_required_for_state_changes</code> , <code>test_foreign_origin_rejected</code> , <code>test_idle_session_expires</code>
Patient enumeration	No list-all endpoint; identical 403 for unknown and unconsented objects; random refs	<code>test_unknown_and_foreign_objects_look_the_same</code> , <code>test_no_list_all_patients_or_phone_lookup</code>
Insider bulk access	Per-doctor rate limit and >25-patients/hour anomaly denial	<code>test_bulk_access_anomaly</code>
Consent withdrawal not honoured	Consent re-queried on every request, including evidence view time	<code>test_expired_consent_denied</code> , <code>test_evidence_view_requires_source_scope_and_live_consent</code> ; live PASS
Evidence URL reuse / forwarding	Single-use, 60 s, session-bound token; watermark; no download	<code>test_evidence_token_single_use_short_lived_session_bound</code> , <code>test_no_direct_document_download</code> ; live PASS
Malicious file (malware, active PDF, bombs)	Magic bytes, isolated worker, structural PDF scan, sanitise, scan both copies, fail closed	<code>test_malicious_files_rejected</code> , <code>test_decompression_bomb_rejected</code> , <code>test_pdf_active_content.py</code>
Parser exploit reaching secrets	Worker has no secrets, no DB, no network, CPU/core limits	<code>test_worker_environment_has_no_secrets</code> , <code>test_worker_blocks_network</code>
Storage theft	AES-256-GCM per-document keys; field encryption; separate identity DB	<code>test_documents_and_fields_are_encrypted_at_rest</code> , <code>test_identity_is_separate_from_medical_data</code>
Hallucinated / misread clinical data	Rule-based extraction, quote-or-reject, states, doctor confirmation	<code>test_unanchored_extraction_is_rejected</code> , <code>test_never_invents_frequency_or_duration</code>

THREAT	JEEVAFLOW DEFENSE	EVIDENCE
Document conflict	Conflict detection with both sources preserved; never auto-resolved	<code>test_conflict_detection_and_rejection</code> ; live check
Audit tampering / truncation	Hash chain + HMAC, append-only triggers, anchored head	<code>test_audit_chain_verifies_and_detects_tampering</code> , <code>test_audit_detects_deleted_tail</code> ; live tamper demo
PHI leaking into logs	Allow-listed log events and fields; tracebacks stripped; no input echo	<code>test_no_phi_in_logs</code> , <code>test_validation_errors_do_not_echo_input</code>
Injection / XSS	ORM parameters; JSON API with CSP <code>default-src 'none'</code> ; React escaping	<code>test_sql_injection_attempts_are_inert</code> , <code>test_xss_payload_is_data_not_markup</code>
Data over-retention	90-day crypto-shredding; unverified uploads shredded on transaction expiry	<code>test_expired_unverified_transaction_documents_are_shredded</code> , <code>test_delete_document_destroys_key_object_and_rows</code>
Demo controls exposed publicly	Demo routes require demo mode, loopback, local Host and no proxy headers	<code>test_demo_controls_refuse_proxied_requests</code>

27 Security Design Principles

#	PRINCIPLE	HOW IT SHOWS UP IN THE CODE
1	Defense in depth	Twelve independent layers (Section 31); e.g. a malicious PDF must pass validation, worker inspection, the structural scan, sanitisation and the scanner, and is then still only quarantined
2	Least privilege	Admin and auditor cannot read clinical data; doctors see only consented patients and scopes; the worker gets no secrets, DB or network
3	Explicit consent	Patient-chosen doctor, scopes and duration; no implied consent from sending a message
4	Separation of communication and data storage	WhatsApp carries generic templates only; documents live encrypted in the vault; identities in a separate database
5	Strong authentication	Patients: transaction-bound OTP. Staff: script password + TOTP with replay protection, lockout and idle timeout
6	Source provenance	Quote-or-reject; every value has quote, page, bbox, confidence and document hash
7	Auditability	Every security-relevant action is chained, signed and verifiable by an independent role
8	Revocability	Consent revocation and “end now” take effect on the next request; documents can be crypto-shredded on demand
9	Data minimisation	Scope-limited views; references instead of identities; no PHI in logs or audit; retention limits; image metadata stripped
10	Human / doctor review	Everything starts in REVIEW; conflicts are never resolved and loops never closed automatically; the doctor is the final gate

28 What JeevaFlow Does Not Do

JeevaFlow organises and secures *documented* information for *authorized* clinical review. It does **not**:

- diagnose patients or interpret whether a value is good or bad;
- replace doctors or make final clinical decisions;
- prescribe, recommend, or independently change medication;
- decide which of two conflicting values is correct;
- close a care loop without a named, MFA-verified doctor;
- treat WhatsApp as the secure medical-data vault (no clinical content is sent over it);
- send documents or extracted data to any external AI service;
- claim regulatory certification. It has not been assessed or certified under HIPAA, DPDP or any other framework, and the README states that these technical controls do not by themselves make it compliant.

29 Limitations / Development Status

This section is intentionally complete. JeevaFlow is a **hackathon MVP** running in a **local development environment** with **synthetic data only**. Do not use it with real patient data without the production replacements below and an independent security review.

29.1 Demo-grade components (labelled as such in the product)

AREA	CURRENT STATE	PRODUCTION REQUIREMENT
Key management	KEK loaded from an environment variable (<code>LocalKeyProvider</code>)	Cloud KMS / Key Vault / HSM behind the existing <code>KeyProvider</code> interface
Malware scanning	Heuristic engine in this environment (ClamAV not installed)	ClamAV or a managed scanning service
Worker isolation	Separate process with in-process egress block	Container/VM with no network and a read-only root
Rate limiting	In-memory, per process (resets on restart; observed during this audit)	Shared limiter (e.g. Redis)
Database	SQLite with field-level encryption; the database files themselves are not encrypted; additive schema upgrades, no migration framework	SQLCipher or a managed database with encryption at rest and least-privilege accounts
Transport	Local HTTP	TLS at the edge, <code>JEEVAFLOW_ENV=production</code> (HTTPS, Secure cookies, demo mode and API docs disabled)
Frontend	No CSP of its own in dev	Serve with a strict CSP header
Audit anchor	Chain head anchored in the identity DB	WORM storage or external timestamping

29.2 WhatsApp status

The Twilio integration (signature verification, REST media download/deletion, freshness check, notifications) is implemented and unit-tested with mocks. **For this audit no Twilio credentials were configured**. The demo used the built-in simulator, which signs messages locally and sends them through the real webhook code. Media download is replaced by a local synthetic file, and media deletion, freshness and outbound notifications are reported `SIMULATED`. A live end-to-end run through Twilio's WhatsApp sandbox was **not verified** in this audit. Also:

- No restricted Twilio API key was configured.
- Webhook processing is synchronous, so very large scanned PDFs may approach Twilio's 15-second timeout.

29.3 Functional limitations

- **Extraction scope:**
 - Lab tests: five (HbA1c, glucose, creatinine, LDL, blood pressure).
 - Instruction verbs: five families (repeat, follow-up, review, refer, recheck).
 - Medicines: a 20-entry formulary.
 - Not reliably understood: free-text notes, unusual table layouts and handwriting.
- **Dates:** read day-first (DD/MM/YYYY). If no report date is found, the date received is used.
- **OCR:** quality depends on Tesseract and the image. The `RETAKE` gate refuses only clearly unreadable images.
- **Consent model:**
 - One purpose (`CLINICAL_REVIEW`).
 - No doctor-initiated request flow.
 - The per-consent `retention_days` field is informational; document retention uses the global 90-day setting.
- **Identifiers:** observation, commitment and evidence IDs in the doctor API are sequential integers. They are protected by authorization and uniform 403s, but they are not opaque.
- **Staff accounts:**
 - Four seeded synthetic accounts.
 - No staff enrolment or MFA-reset UI.

- Demo TOTP secrets are derived from a demo seed.
- **Watermarking:** discourages and attributes leaks; it cannot prevent screen photography.
- **Audit log retention:** unbounded; no archival policy.
- **Demo surfaces:**
 - The demo phone view shows OTP codes so the demo can run without a phone.
 - Demo routes are restricted to localhost and demo mode.
- **Not performed:** penetration test, threat-modelling workshop, formal privacy impact assessment, compliance assessment, load or availability testing.

29.4 Security assumptions

- The host running the backend is trusted and not compromised.
- Secrets in `.env` are protected (generated by `init_secrets`, file mode 600, git-ignored).
- Twilio's signing token is kept secret.
- Staff devices and authenticator apps are under staff control.
- Users access the portal over a secure network in production.

30 Future Extensions

Everything in this section is planned or proposed. None of it is implemented today.

AREA	FUTURE WORK
Messaging	Production WhatsApp Business sender with a restricted API key; asynchronous webhook processing (queue) for large documents
Identity	Enterprise identity integration (OIDC/SAML SSO) for staff, with MFA policy enforcement and staff enrolment / recovery flows
Keys	Cloud KMS or HSM-backed KEK via the existing <code>KeyProvider</code> seam; key rotation
Malware	ClamAV or a managed multi-engine scanning service; content disarm and reconstruction
Platform	Containerised worker with no network and a read-only root; managed database with encryption at rest; Redis rate limiting; migrations
Compliance	Formal compliance assessment (e.g. DPDP, HIPAA where applicable), privacy impact assessment, independent penetration test
Operations	Deployment hardening, observability (metrics, alerting on audit events and anomalies), disaster recovery, high availability, backup with key separation
Audit	External anchoring of the audit-chain head (WORM storage / timestamping), audit archival policy
Extraction	Broader test and medication coverage, table-aware parsing; any ML-assisted extraction would remain quote-or-reject and doctor-confirmed
Analytics	Privacy-preserving, aggregate analytics; federated learning only as a possible <i>future</i> analytics architecture, subject to consent and governance

31 Final Security Summary

- 1 **Communication security**
Generic replies only; media deleted from Twilio; unknown numbers never become patients
- 2 **Webhook verification**
X-Twilio-Signature (fails closed), replay, account, freshness, rate limits
- 3 **Patient authentication**
Transaction-bound OTP: HMAC stored, 5 min, single use, 5 attempts + logout
- 4 **Consent**
Patient-chosen doctor, scopes, 1/7/30-day expiry; revoke / end now
- 5 **Document security**
Validation, isolated worker, structural PDF scan, sanitise, malware scan, quarantine
- 6 **Encryption / vault**
AES-256-GCM per document (envelope), field-level encryption, separate identity DB
- 7 **Provenance**
Quote-or-reject; every fact has quote, page, bbox, confidence, document hash
- 8 **Doctor MFA**
script password + TOTP with replay block; logout; server-side sessions + CSRF
- 9 **Authorization**
Deny-by-default roles + consent + scope checked on every request; generic 403
- 10 **Evidence controls**
Single-use 60 s session-bound token, watermarked render, no download
- 11 **Audit chain**
SHA-256 hash chain + HMAC, append-only triggers, anchored head, auditor verify
- 12 **Retention**
Crypto-shredding after 90 days and for unverified uploads; delete = destroy key first

All twelve layers are implemented in the repository; demo-grade parts are named in Section 29.

Figure 31.1 — Defense in depth: twelve layers, all present in the repository. Demo-grade parts of individual layers are listed in Section 29.

JeevaFlow's security model rests on one separation and three gates:

- **The separation.** WhatsApp carries the interaction; JeevaFlow holds the data.
- **The patient gate.** A transaction-bound OTP, then explicit, scoped, revocable consent.
- **The clinician gate.** Password + TOTP, then consent and scope checked on every request.
- **The accountability gate.** Every action is recorded in a tamper-evident chain that an independent auditor can verify.

Between the gates, every document is isolated, sanitised, scanned, encrypted with its own key and kept only as long as necessary. Every clinical value carries its source. These properties were verified in code, by 156 automated backend tests and 8 frontend tests, and by a 13-step live run in the development environment. The production replacements in Section 29 remain to be done.

Appendix A. Audit Method and Evidence

Scope reviewed (2 October 2026):

AREA	WHAT WAS REVIEWED
Backend application	<code>backend/app/</code> (all modules, routers and security package)
Tests and scripts	<code>backend/tests/</code> , <code>test_api.py</code> , <code>seed_demo.py</code> , <code>run_demo.sh</code>
Configuration	<code>requirements.txt</code> , <code>.env.example</code>

AREA	WHAT WAS REVIEWED
Frontend	<code>frontend/src/</code> pages and components, <code>package.json</code> , <code>vite.config.ts</code>
Documentation	<code>README.md</code> , <code>docs/</code>
Demo data	the four synthetic PDFs in <code>data/demo/</code>

`backend/.env` was **not** opened.

Execution:

1. Ran the backend suite (`pytest`): 156 passed.
2. Ran the frontend suite (`vitest`): 8 passed.
3. Started an **isolated** backend with freshly generated random keys, temporary databases and vault, no Twilio credentials, and a placeholder synthetic phone number.
4. Ran `test_api.py` : 13/13 passed.
5. Ran an additional verification script covering conflicts, loops, roles, tamper/restore, retention and security status.
6. Captured all screenshots with headless Chrome against that instance. One-time codes and transaction references were masked before capture.

Excluded from this document: passwords, TOTP seeds or codes, API keys, Twilio credentials, session or evidence tokens, encryption keys, and real phone numbers. Screenshots show only synthetic names and opaque references (`pt_...` , `usr_...`) generated by the throwaway instance. The masked phone shown (`+91*****00`) belongs to a placeholder number used only for this run.

Team attribution: the team name *Team Geek* was supplied by the project team. It does not appear in the repository.

Report assets:

ASSET	LOCATION	PRODUCED BY
Diagrams	<code>docs/report/assets/diagrams/*.svg</code>	<code>docs/report/build/diagrams.py</code>
Screenshots	<code>docs/report/assets/screenshots/</code>	headless Chrome against the isolated instance
PDF	<code>JeevaFlow_Security_Features_Report.pdf</code>	<code>docs/report/build/build_pdf.mjs</code> , rendered from this Markdown